Shared Light

Collaborative Projective XR

Projection Lantern Edition

A critical design, calibration algebra, and build plan
for artists and computer scientists

Project Synchronicity

10 November 2026 / Design revision 3 / illustrated

The project in one minute

People draw and perform into one shared spatial canvas. Fixed projectors and handheld or parked projection lanterns put that canvas onto real surfaces. Cameras work out where devices and objects are. A measured, barcode-covered object—the Fulcrum Frustum (FF)—gives them a common ruler.
The key separation is **what exists** (the artwork), **where it is** (its attachment to the room or an object), and **who is shining light on it** (each projector's current viewpoint). Moving a lantern's carrier changes the last part, not the artwork's identity.

Reading routes. For a presentation: Sections 1, 2, 12, and 14. For implementation: follow the equations in order, including the timing and message contracts. Blue explanation boxes give the intuition; the surrounding equations give the precise recipe.

Scope. This is an audited synthesis of nine supplied Word documents, not a claim that every proposed capability already exists. It specifies the geometry needed for static and moving cameras/projectors, initial calibration, live relocalization, shared content, and bounded prediction. It does not provide a finished SLAM system, trained neural model, certified optical device, or measured performance guarantee.

Deliverable. This file is self-contained: upload this `.tex` alone to Overleaf, choose pdfLaTeX, and compile. No external images are needed. The older `ff_calibration.tex` is deliberately left unchanged.

Contents

1 Experience First: A Room That People Can Paint Together **3**

2 Audit: Keep the Vision, Remove the Impossible Promises **4**

3 The Language: Rulers, Places, and Directions **6**

4 The Lens Recipe: From a Point to a Pixel **8**

5 What the FF Measures, and What PnP Actually Solves **9**

6 The Shared World: Initial Registration and Moving Anchors **11**

7 Projectors: Fixed Lights and Gimballed Projection Lanterns **13**

8 Surfaces, Depth, Rays, and the Limits of a Canvas **16**

9 Time: Calibrate When the Light Lands, Not When a Packet Arrives **18**

10 Tracking, IMUs, Dropouts, and Recovery **20**

11 Collaborative Canvas and the Spatial Pipe **23**

12 Initial and On-the-Go Calibration: Operational Recipes **25**

13 Joint Refinement and an Honest Validation Plan **28**

14 Build Plan: Color-Coded Tasks and Evidence of Done **29**

15 Presentation Cheat Sheet **34**

A Source Register and Traceability **34**

1 Experience First: A Room That People Can Paint Together

Imagine a glowing line on a pedestal. Alice draws its next segment with a phone. Bob's carried lantern illuminates one side; a ceiling projector illuminates the other. A person walking away should not erase the line. Moving the pedestal should move only artwork intentionally attached to it. Losing tracking should fade the affected light, not send a misplaced bright image across the room.

1.1 Three different kinds of shared artwork

- **Room-locked:** a line belongs to the wall or stage and stays there when the FF moves.
- **Object-locked:** a line belongs to the FF, a prop, or a costume, and follows that object's motion.
- **View-dependent:** an anamorphic illusion looks correct from a selected eye position. It will not generally look correct to everyone.

Ordinary projection is shared visible light, not a private image for each eye. Two viewers can share a painted surface; they cannot in general share one perfect perspective illusion from different viewpoints.

1.2 Separate device roles from device motion

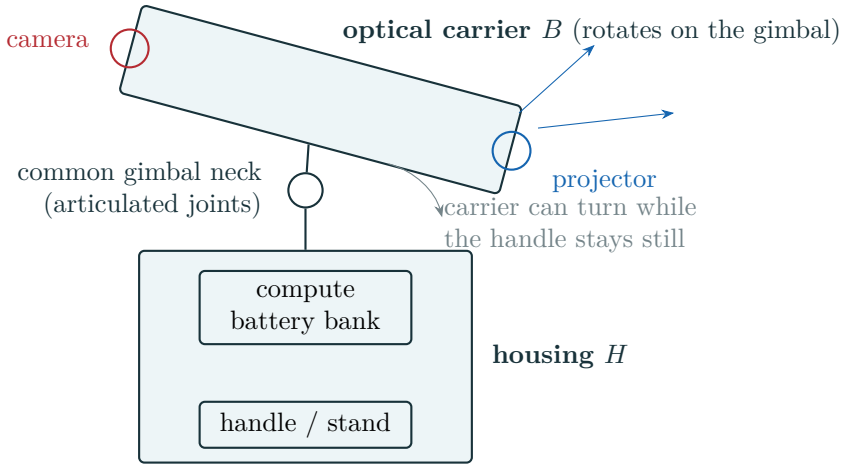
A *producer* creates measurements or artwork. An *observer* renders a view. A *hybrid* does both. These are software roles, not hardware species: a phone can draw and view; a projector emits light but does not sense the room unless it has an associated camera or another tracker.

Device	What is stable?	What changes live?	Required evidence
Static webcam	Lens model, mount pose if untouched	Image and visible objects	Known room landmarks to detect a bump
Moving webcam/phone	Lens model if optical settings stay fixed	Six-dimensional pose	Known markers, VIO/SLAM, or external tracking
Static projector	Lens model and mount pose	Image/content	Initial pixel-to-surface calibration; bump checks
Moving projector / lantern optical carrier	Lens model, rigid carrier offsets and gimbal joint model	Six-dimensional carrier pose at light-emission time	Tracked carrier, or tracked housing plus measured joint angles
Depth camera	Depth and color models; rigid color/depth offset	Depth samples and possibly sensor pose	Metric units, valid depth, registration and timestamps

1.3 Meet the projection lantern

The baseline moving light is a handheld or parked *projection lantern*, not a head-worn device. Its vented outer *housing* carries the handle or stand, the compute and the battery bank below a common gimbal neck. The gimbal holds one rigid *optical carrier* bearing the camera and projector together, with unobstructed apertures for both lenses, cable service loops across the joints, and a

transport lock for packing. Moving mass for compute and battery stays in the housing; the final mechanical balance depends on the chosen payload and testing.



Schematic component layout, not an optical ray diagram: both the camera and the projector face the working area. The camera and projector move **together** on the carrier; neither keeps a fixed offset to the housing while the gimbal turns. A parked lantern whose joints are locked behaves like a static projector; a parked housing with moving joints does not. Figure 1 sketches the three cooperating loops that keep such a lantern honest.

A good first demonstration

Start with one room-locked surface, one fixed projector, one moving camera–projector pair on a carried lantern's gimballed carrier, and a fixed FF. Prove agreement on that surface before adding a second moving lantern, smoke, or neural prediction.

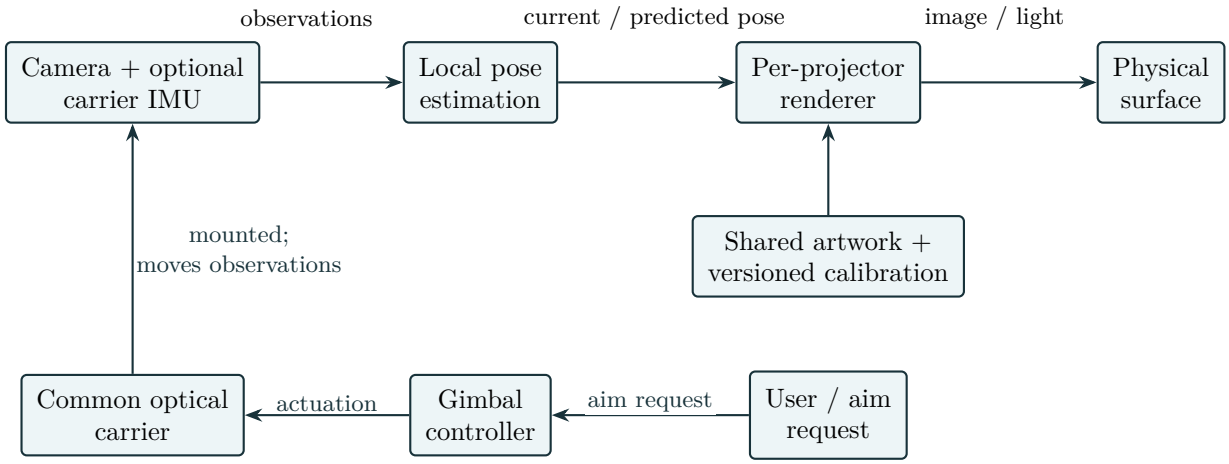


Figure 1: The gimbal moves the optical platform; sensing estimates its actual pose; rendering places the artwork. The shared pipe supplies artwork and calibration, not magical localization. This is conceptual control/data flow, not electrical wiring.

2 Audit: Keep the Vision, Remove the Impossible Promises

Source keys S1–S9 refer to the exact supplied filenames in Section A. These documents are design inputs, not independent experimental evidence. Their placeholder references such as `[cite: 2]` are not traceable publications and are not retained as proof.

Verdict	Claim in the inputs	Correction / consequence
Critical / S6	Detect a marker ID and fetch the master transform; no local geometry solve.	ID gives identity, not six-dimensional pose. A device still needs calibrated image corners and a pose solve, an existing local map aligned to the world, or an external pose measurement. A server may solve remotely, but the measurement and time alignment do not disappear.
Critical / S5	An FF observation distinguishes an FF move from a bumped camera.	It measures their relative pose only. A trusted fixed landmark, fixed camera, or independently world-aligned tracker is required to attribute motion. All devices and the FF can otherwise drift together.
Critical / S2–S4	Interpolation and NPU prediction eliminate latency and guarantee 60 Hz.	Interpolation needs bracketing samples and therefore usually a delay. Extrapolation estimates the future and can be wrong. A worker protects the render thread but cannot guarantee GPU deadlines or light-emission timing.
Critical / S1, S8	High-intensity/IR countermeasures aimed at eyewear; a shutter guarantees safety.	Exclude targeted sensor-blinding from the artistic system. Do not aim at eyes or faces. Black pixels, software masks, and generic privacy/welding films are not certified optical safety shutters. A handheld beam can still sweep a bystander; assess the complete optical system before use around people.
Major / S9	FF height is 100 mm; rank-three face normals guarantee a positive-definite pose Hessian and no depth ambiguity.	The repository's measured v1 height is 42.5 mm. Normals alone do not prove pose observability. Visible corner layout, image size, viewing angle, intrinsics and noise determine conditioning. A planar target can yield a valid calibrated pose; non-planarity is helpful, not magical.
Major / S2–S5	Stage FF is 5 m wide, so scale and occlusion are solved.	This is a proposed different artifact, with no supplied height or measured marker layout. It needs its own surveyed model. Larger targets can help pixel coverage but do not remove occlusion or motion blur.
Major / S2–S5, S8	Translation in $[R \mid t]$ is the camera's world position; pointing direction determines orientation.	For world-to-camera extrinsics the center is $-R^\top t$. A direction omits roll: give a full rotation or a nonparallel up direction.
Major / S7	`solvePnP` estimates lens distortion as well as pose.	Standard PnP takes K and distortion as inputs and estimates rotation and translation. Intrinsic/distortion calibration is a separate, sufficiently excited multi-view optimization.
Major / S2–S6	One nearby marker is equivalent to the full non-planar target.	A known square can support pose, but is planar, often ambiguous/noisy, especially small or front-facing. Use candidate poses, positive depth, temporal/IMU evidence and uncertainty; fall back to more landmarks.
Major / S8, S9	One homography aligns all 3D artwork from any moving viewpoint.	A homography is exact only for a plane in undistorted pinhole coordinates (or special pure-rotation view changes). Multiple depths need per-surface maps or full 3D rendering and depth tests.

Verdict	Claim in the inputs	Correction / consequence
Major / S5, S6	An IMU/Myo supplies a world-space ray origin; a phone's known position alone can recalibrate a camera.	An IMU measures rotation rate and specific force, not absolute position. A ray needs a known origin. A phone must expose enough measured visual geometry with known pose; an invisible network pose is not a correspondence.
Major / S1, S8	Beams meeting in smoke form a stable free-floating 3D canvas.	Haze scatters along entire beams; rays continue beyond their crossing. Conventional projectors do not selectively illuminate arbitrary isolated voxels. Treat smoke as a beam sculpture, not a volumetric display.
Major / S1, S8	Two projectors automatically double usable brightness; overlap is fixed at a sweet-spot distance.	Irradiances add only where footprints meet. Calibrated rendering can align on known surfaces at many distances, but overlap, focus, falloff, black level, occlusion and eye safety still constrain the result.
Major / S1, S8	A 65/100 W bank and generic 12 V trigger guarantee two-projector operation.	Total wattage does not specify per-port voltage/current, simultaneous profiles, startup peaks or thermal derating. Check exact modules, cable rating and negotiated profiles; do not assume 12 V PD exists.
Unproven / S1, S8, S9	Millimetre/sub-millimetre alignment and a fixed affordable bill of materials.	These are targets, not measurements. Specify range, pixel coverage, static/dynamic error and percentiles; include optics, cooling, mounts, safety, cables, power conversion and local price verification.
Keep / S2–S8	Local rendering, a shared spatial pipe, asynchronous tracking and simple vector art.	Strong architecture once every transform has a direction, timestamp, revision and uncertainty, and the system defines failure behavior. Simple visuals reduce visual sensitivity but do not correct bad geometry.

3 The Language: Rulers, Places, and Directions

A coordinate frame is a labelled ruler

A frame says where zero is and which directions count as right, forward, and up. The same physical point has different numbers when measured from the room, the FF, or a camera. A transform translates between those rulers; it does not move the physical point.

3.1 Conventions fixed before any equation

Use column vectors, right-handed frames, metres, seconds and radians. The existing repository uses millimetres for FF geometry/extrinsics: **convert at the interface**, not halfway through a matrix product. Pixel coordinates are in pixels, with u to the right and v down.

The room frame is W, chosen once per session; its $+z$ is up. The FF frame is F, centered on its base, with $+z$ up. C_i names camera i, D_i a depth camera, P_j projector j, B a rigid moving body—in the lantern design this is the gimballed *optical carrier*—H the lantern *housing* frame when a lantern is present, and M_k a known marker. An optical device's axes follow OpenCV: $+x$ right, $+y$ down, $+z$ forward. The carrier's axes may differ from the housing's; document both. Where the letter H appears as a filter observation Jacobian, context keeps it distinct from the

housing frame.

Symbol	Meaning and shape
t, Δt	Time and elapsed time, in seconds. A bold $\boldsymbol{t}$ below is a translation vector, not time.
$\boldsymbol{x}_A$, $\bar{\boldsymbol{x}}_A$	A point measured in frame A, a 3×1 vector, and the same point with a final 1 appended, a 4×1 vector.
$T_{A \leftarrow B}(t)$	A 4×4 transform taking coordinates *from* B *to* A. Read the arrow literally. A, B here are any named frames.
$R_{A \leftarrow B}$, $\boldsymbol{t}_{A \leftarrow B}$	Its 3×3 rotation and 3×1 translation.
I, $\mathbf{0}$, $(\cdot)^\top$	Identity matrix, zero vector/matrix, and transpose; dimensions follow the surrounding equation.
$\boldsymbol{c}_W$, $\boldsymbol{p}_W$	An optical device's center and unit forward direction, both expressed in the room. These are not extrinsic translation.
$w_V, h_V, K_V, \boldsymbol{d}_V$	Device V's width, height, intrinsic matrix, and distortion coefficients. V can be any camera or projector.
$\boldsymbol{q} = (u, v)^\top$, $\bar{\boldsymbol{q}} = (u, v, 1)^\top$	A measured image pixel and its homogeneous form.
$\pi_V(\boldsymbol{x}_V)$	The complete projection recipe for device V: divide by depth, distort, then apply intrinsics.
$\|\boldsymbol{a}\|$, $\boldsymbol{a} \times \boldsymbol{b}$	Euclidean length and three-dimensional cross product.
$\hat{}$, $^-$, $^+$	Estimated quantity, estimate before a measurement correction, estimate after correction.
Σ, P, Q, N	Measurement covariance, state-error covariance, process-noise covariance, and measurement-noise covariance in the filter. Projector symbols have an indexed P_j, not an unindexed covariance P.

Additional symbols are introduced next to their first equation. A covariance describes uncertainty and correlations, not a generic "confidence score."

3.2 The one transform rule to remember

A rotation turns a direction; a translation shifts a point:

$$\boldsymbol{x}_A = R_{A \leftarrow B} \boldsymbol{x}_B + \boldsymbol{t}_{A \leftarrow B}, \qquad \bar{\boldsymbol{x}}_A = T_{A \leftarrow B} \bar{\boldsymbol{x}}_B, \qquad T_{A \leftarrow B} = \begin{bmatrix} R_{A \leftarrow B} & \boldsymbol{t}_{A \leftarrow B} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \tag{1}$$

A proper rotation has $R^\top R = I$ and $\det R = +1$. These rotations form the set $SO(3)$; the rigid transforms form $SE(3)$. Those names mean "legal rotations" and "legal rotation-plus-translation recipes," not additional hardware.

To chain transforms, match the inner frame names. To reverse a transform, undo the translation in the rotated coordinates:

$$T_{A \leftarrow C} = T_{A \leftarrow B} T_{B \leftarrow C}, \qquad T_{B \leftarrow A} = T_{A \leftarrow B}^{-1} = \begin{bmatrix} R^\top & -R^\top \boldsymbol{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \tag{2}$$

Here $R = R_{A \leftarrow B}$ and $\boldsymbol{t} = \boldsymbol{t}_{A \leftarrow B}$. Products are evaluated right to left. At live time, all dynamic factors must refer to the **same physical time**, not simply their latest arrival times.

3.3 Where a camera really is

For world-to-camera extrinsics $R = R_{C\leftarrow W}$, $\boldsymbol{t} = \boldsymbol{t}_{C\leftarrow W}$, the camera center and its pointing direction in the world are

$$\boldsymbol{c}_W = -R^\top \boldsymbol{t}, \qquad \boldsymbol{p}_W = R^\top (0,0,1)^\top. \tag{3}$$

Why? The camera's own center has coordinates $\mathbf{0}$ in camera space. Substitute that into Equation (1) and solve backwards. Its translation $\boldsymbol{t}$ instead tells us where the *world origin* appears in the camera.

For a human-friendly "look at" pose, specify center $\boldsymbol{c}_W$, unit forward $\boldsymbol{f}$ and an approximate world-up vector $\boldsymbol{u}$ that is not parallel to $\boldsymbol{f}$. Define right $\boldsymbol{r}$ and down $\boldsymbol{b}$:

$$\boldsymbol{r} = \frac{\boldsymbol{f} \times \boldsymbol{u}}{\|\boldsymbol{f} \times \boldsymbol{u}\|}, \quad \boldsymbol{b} = \boldsymbol{f} \times \boldsymbol{r}, \quad R = \begin{bmatrix} \boldsymbol{r}^\top \\ \boldsymbol{b}^\top \\ \boldsymbol{f}^\top \end{bmatrix}, \quad \boldsymbol{t} = -R\boldsymbol{c}_W. \tag{4}$$

If looking straight up/down, choose a different nonparallel reference-up. That choice fixes roll; pointing alone never does.

4 The Lens Recipe: From a Point to a Pixel

4.1 Perspective, intrinsics and distortion

Let $\boldsymbol{x}_V = (X, Y, Z)^\top$ be a point in device V. Only $Z > 0$ is in front of the lens. Normalize by depth: $x = X/Z$, $y = Y/Z$. These x, y are dimensionless. Let f_x, f_y be focal lengths in pixels, c_x, c_y the principal point, and γ pixel-axis skew (normally fixed to zero):

$$K_V = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{5}$$

For the common five-coefficient OpenCV lens model, store $\boldsymbol{d}_V = (k_1, k_2, p_1, p_2, k_3)$ *in that order.* With squared radius $r^2 = x^2 + y^2$ and radial factor a, the full recipe is

$$a = 1 + k_1 r^2 + k_2 r^4 + k_3 r^6, \tag{6}$$

$$x_d = xa + 2p_1 xy + p_2(r^2 + 2x^2), \tag{7}$$

$$y_d = ya + p_1(r^2 + 2y^2) + 2p_2 xy, \tag{8}$$

$$\pi_V(X, Y, Z) = \begin{bmatrix} f_x x_d + \gamma y_d + c_x \\ f_y y_d + c_y \end{bmatrix}. \tag{9}$$

Fisheye lenses need a matching fisheye model; do not reinterpret their coefficients as these five numbers.

Perspective is a divide, not a mysterious squash

A point twice as far away has half the image displacement from the principal point. The matrix sets the ruler in pixels; distortion bends that ideal ruler. Use the same recipe for solving and for drawing the overlay.

With no distortion, the compact homogeneous equation is

$$Z\bar{\boldsymbol{q}} = K_V \begin{bmatrix} R_{V\leftarrow W} & \boldsymbol{t}_{V\leftarrow W} \end{bmatrix} \bar{\boldsymbol{x}}_W. \tag{10}$$

The multiplier is device-space depth for this normalization. The final division by the third coordinate is essential. With distortion, use Equation (9), not this matrix alone.

4.2 Resolution changes, cropping and automatic optics

For a pure coordinate resize $u' = s_x u$, $v' = s_y v$,

$$K' = \operatorname{diag}(s_x, s_y, 1)K, \qquad \boldsymbol{d}' = \boldsymbol{d}. \tag{11}$$

For cropping by offsets u_0, v_0 before resizing, use $c'_x = s_x(c_x - u_0)$, $c'_y = s_y(c_y - v_0)$. Some resamplers map pixel centers as $u' = s_x(u + 1/2) - 1/2$; then the same offset belongs in c'_x. Record the actual sampling convention and full image preprocessing map. Rotation, mirroring, stabilization, digital zoom and keystone are not automatically described by simple focal-length scaling.

Lock optical zoom/focus where possible. Disable auto-keystone, obstacle avoidance, auto-fit, and changing digital stabilization for a calibrated projector. A calibrated, fixed warp can be explicitly modeled, but an unknown changing warp invalidates the model. Recheck intrinsics after an optical focus/zoom change; a digital focus command is not a pose update.

4.3 Example 1: A deliberately tiny 32×16 image

This is an arithmetic aid, **not** enough resolution for practical FF tracking. Choose zero distortion and

$$K = \begin{bmatrix} 16 & 0 & 16 \\ 0 & 16 & 8 \\ 0 & 0 & 1 \end{bmatrix}, \qquad K^{-1} = \begin{bmatrix} 1/16 & 0 & -1 \\ 0 & 1/16 & -1/2 \\ 0 & 0 & 1 \end{bmatrix}. \tag{12}$$

Our continuous image coordinates have bounds $[0, 32) \times [0, 16)$. The principal point is chosen as $(16, 8)$; real principal points are measured, not forced to the midpoint. Horizontal field of view is $2\arctan(32/(2 \cdot 16)) = 90°$; vertical is about $53.13°$, *not* $90°$.

For $\boldsymbol{x}_C = (0.25, 0.125, 1)$ metres, normalized coordinates are $(0.25, 0.125)$, so $\boldsymbol{q} = (20, 10)$ pixels. Conversely $(20, 10)$ identifies the ray $K^{-1}(20, 10, 1)^\top = (0.25, 0.125, 1)^\top$, not a unique point: depth or a surface is still missing.

5 What the FF Measures, and What PnP Actually Solves

5.1 Use the built artifact, not a convenient ideal pyramid

For the repository's **v1 physical FF**, base edge $b = 0.155$ m, top edge $a = 0.075$ m, height $h = 0.0425$ m. Top marker ID 10 is 0.055 m square; side markers 11–14 are 0.050 m. The dictionary is `DICT_4X4_50`; IDs 11,12,13,14 label $+y, +x, -y, -x$, respectively.

Base vertices are $(\pm b/2, \pm b/2, 0)$; top vertices are $(\pm a/2, \pm a/2, h)$. Let inset $s = (b - a)/2 = 0.040$ m. The slope angle *above horizontal* is $\arctan(h/s) \simeq 46.736°$. For example the outward $+y$ side normal is

$$\boldsymbol{n}_{+y} = \frac{(0, h, s)^\top}{\sqrt{h^2 + s^2}} \simeq (0, 0.72820, 0.68536)^\top. \tag{13}$$

The top marker's corners, in decoded top-left, top-right, bottom-right, bottom-left order, are

$$\begin{bmatrix} -0.0275 & 0.0275 & 0.0425 \\ 0.0275 & 0.0275 & 0.0425 \\ 0.0275 & -0.0275 & 0.0425 \\ -0.0275 & -0.0275 & 0.0425 \end{bmatrix} \text{ m}. \tag{14}$$

Rows list points; the transform equations still use column vectors. Use all side-marker corners from `fulcrum_spec.json`, divided by 1000, rather than inferring sticker positions from face dimensions.

The file notes an assumed, unmeasured 4 mm sticker gap and has an older "about 45 mm" height note: re-survey the real object before asserting sub-millimetre accuracy. The measured block's current height is 42.5 mm.

The proposed 5 m stage object in S2–S5 has no complete supplied survey. It is not a scaled copy of this artifact unless explicitly built and measured as such. Also distinguish these *document revisions* from the repository's `ffv2_spec.json`, which describes a separate painted tracking solid.

5.2 The correspondence contract

For each visible marker corner i, take the known point $\boldsymbol{x}_{F,i}$ from the model and its detected image point $\boldsymbol{q}_{C,i}$. Match marker identity *and decoded corner order*. Knowing the printed size alone is insufficient if its placement on the FF is unknown. Never sort corners only by screen x or y.

Let $\mathcal{I}$ be the accepted corner indices. PnP estimates a pose $T_{C\leftarrow F}$ with fixed $K_C, \boldsymbol{d}_C$:

$$(\hat{R}, \hat{\boldsymbol{t}}) = \underset{R\in SO(3),\, \boldsymbol{t}}{\arg\min} \sum_{i\in\mathcal{I}} \rho\Big(\boldsymbol{e}_i^{\top} \Sigma_i^{-1} \boldsymbol{e}_i\Big), \qquad \boldsymbol{e}_i = \boldsymbol{q}_{C,i} - \pi_C(R\boldsymbol{x}_{F,i} + \boldsymbol{t}). \tag{15}$$

Here $\boldsymbol{e}_i$ is a two-pixel-component residual; Σ_i is its 2×2 covariance; ρ is a robust loss that reduces outlier influence. This is a *desired robust objective*, not a statement that ordinary `solvePnP` automatically implements Huber loss or covariance weighting. Use a robust wrapper/RANSAC to reject bad correspondences, then refine and re-evaluate all accepted observations.

Outputs. OpenCV returns a rotation vector `rvec` and translation `tvec`, taking FF points *into camera space*. Convert rotation using `cv2.Rodrigues`. If $\boldsymbol{\theta}$ is a rotation vector, $\theta = \|\boldsymbol{\theta}\|$ its angle and $\boldsymbol{a} = \boldsymbol{\theta}/\theta$ its axis, then

$$R = I + \sin\theta\,[\boldsymbol{a}]_\times + (1-\cos\theta)[\boldsymbol{a}]_\times^2, \qquad [\boldsymbol{a}]_\times = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}. \tag{16}$$

Use the zero-angle limit $R = I$. A quaternion is another orientation representation; state its component order and normalize it. Do not average rotation matrices or quaternion components as if they were positions.

5.3 Ambiguity, conditioning, and honest quality

A single square supplies four coplanar corners and can support pose with known intrinsics; it may yield competing or weakly separated solutions. IPPE-style planar candidate evaluation is useful. Non-coplanar visible corners help, but a solver's minimal point count is not a production-quality acceptance rule.

Require positive depths, enough image spread, plausible motion, and agreement with independent observations. The offline repository registration requests at least three FF markers; its TD live path can use two, or one during short-term tracking with a prior. Those paths do not have equal certainty. Do not promote a one-marker continuation to a fresh global registration.

For local pose perturbations $\delta\boldsymbol{\xi}$ (three translation and three rotation components), let J be the Jacobian of residuals with respect to those six components and let W_e be their inverse-covariance weighting. Under a locally Gaussian, well-conditioned approximation,

$$\Sigma_\xi \simeq (J^{\top} W_e J)^{-1}. \tag{17}$$

Near-singularity means uncertainty is large; do not silently invert it and claim precision. This local formula omits lens and model error unless those uncertainties are included. Face-normal rank is *not* a proof about $J^{\top} W_e J$.

Report per-view RMS pixel error $e_{\mathrm{RMS}} = \sqrt{\sum_i \|\boldsymbol{e}_i\|^2/|\mathcal{I}|}$, maximum error, marker coverage, age, and independent surface alignment. Mean corner distance and RMS are different metrics. Low in-sample reprojection error alone does not prove correct world anchoring. At focal length 1000 px and distance 5 m, a 50 mm marker is only about 10 px wide before foreshortening: that is a warning, not a reliable room-scale reference.

5.4 Lens calibration is a different optimization

Before live pose tracking, collect diverse views over the lens field, with different tilts, distances, and adequate lighting. Let k index views, i corners, and $R_k, \boldsymbol{t}_k$ each target-to-camera pose. Estimate shared lens parameters along with separate per-view poses:

$$\underset{K,\boldsymbol{d},\{R_k,\boldsymbol{t}_k\}}{\arg\min} \sum_{k,i} \rho\Big(\|\boldsymbol{q}_{ki} - \pi_{K,\boldsymbol{d}}(R_k\boldsymbol{x}_{F,i} + \boldsymbol{t}_k)\|^2\Big). \tag{18}$$

`calibrateCamera` and suitable calibration tools support this class of problem. Priors and held-out views help prevent overfitting. One small FF at one image location cannot reliably constrain all distortion parameters. Lock intrinsics for routine live PnP; motion usually requires a new pose, not a new lens fit.

6 The Shared World: Initial Registration and Moving Anchors

The FF can be a ruler without being the room

At setup, you may declare "the FF's current base is the room origin." Afterwards the room ruler stays put. The physical FF may move. Artwork attached to the FF follows it; artwork attached to the room does not. Someone must keep observing the relation between those two rulers.

6.1 Initial fixed-FF registration

At setup time t_0, hold the FF still and choose $T_{W\leftarrow F}(t_0)$. The simplest choice is I: the room agrees with the FF *at that instant*. For every camera seeing it,

$$T_{W\leftarrow C_i}(t_0) = T_{W\leftarrow F}(t_0)T_{C_i\leftarrow F}(t_0)^{-1}. \tag{19}$$

For a genuinely fixed camera, retain this world pose while its mount remains unchanged. Simultaneous observations of a fixed FF give the camera-to-camera registration

$$T_{C_i\leftarrow C_j} = T_{C_i\leftarrow F}T_{C_j\leftarrow F}^{-1}. \tag{20}$$

Sequential captures are acceptable only while the FF *and both cameras* are still. The term "paired" does not itself mean hardware synchronized.

6.2 A moving FF with an independent fixed reference

Let C_s be a trusted static camera with known $T_{W\leftarrow C_s}$. It observes the moving FF:

$$T_{W\leftarrow F}(t) = T_{W\leftarrow C_s}T_{C_s\leftarrow F}(t). \tag{21}$$

A moving camera C_m also sees the FF at that same time:

$$T_{W\leftarrow C_m}(t) = T_{W\leftarrow F}(t)T_{C_m\leftarrow F}(t)^{-1}. \tag{22}$$

This is the general live-recalibration equation. It requires *independent knowledge* of $T_{W\leftarrow F}(t)$. If only FF-relative artwork matters, a camera–projector pair can operate directly in F without a persistent room frame.

6.3 Why a lone moving FF is not enough

The observation relation is $T_{C\leftarrow F} = T_{W\leftarrow C}^{-1} T_{W\leftarrow F}$. For any common world transform $G(t)$,

$$\big(G(t)T_{W\leftarrow C}(t)\big)^{-1}\big(G(t)T_{W\leftarrow F}(t)\big) = T_{W\leftarrow C}(t)^{-1}T_{W\leftarrow F}(t). \tag{23}$$

The image cannot distinguish these alternatives. This unobservable freedom is often called *gauge freedom*. It is why a low residual cannot tell whether the FF moved, the camera moved, or both moved together.

Provide a surveyed static marker network, a trusted fixed camera, or a VIO/SLAM map aligned to W to preserve room-lock. If all absolute references are lost, publish `relative-only` or growing uncertainty, not a fictional fully tracked room pose. Multiple cameras help only when they connect to independent constraints; a circle of mutually derived estimates does not create new evidence.

6.4 Distributed local maps and marker relays

Suppose a phone tracks in local map L, and that map provides $T_{L\leftarrow C}(t)$. A known world marker M is observed with PnP $T_{C\leftarrow M}(t)$, and its world pose $T_{W\leftarrow M}(t)$ is known. Align the map once, then update as needed:

$$T_{W\leftarrow L} = T_{W\leftarrow M}(t)\big(T_{L\leftarrow C}(t)T_{C\leftarrow M}(t)\big)^{-1}, \qquad T_{W\leftarrow C} = T_{W\leftarrow L}T_{L\leftarrow C}. \tag{24}$$

This is the valid version of "detect and align." It still needs the camera's observation or an existing pose, that camera's own intrinsics, and a known marker pose at the capture time. There is no universal `K_master` suitable for every device.

A tracked phone can serve as a relay only if another camera observes known-size, known-pose geometry attached to it (for example a calibrated marker), at an aligned time. Network position alone is not visible evidence.

6.5 Example 2: A moved FF does not move the room

Use metres and a camera looking straight down from $(0, 0, 1)$:

$$R = \operatorname{diag}(1, -1, -1), \quad T_{C\leftarrow W} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{25}$$

Move the FF without rotation to world position $(0.2, 0.1, 0)$. Then

$$T_{W\leftarrow F} = \begin{bmatrix} 1 & 0 & 0 & 0.2 \\ 0 & 1 & 0 & 0.1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad T_{C\leftarrow F} = \begin{bmatrix} 1 & 0 & 0 & 0.2 \\ 0 & -1 & 0 & -0.1 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{26}$$

The camera's FF-relative center becomes $(-0.2, -0.1, 1)$. Equation (22) still gives its world center $(0, 0, 1)$. A point at the FF origin moves to $(0.2, 0.1, 0)$; a previously room-locked point at $(0, 0, 0)$ stays there.

For the same overhead camera and the FF at the initial origin, the four top-marker pixels using Equation (12) are approximately $(15.54047, 7.54047)$, $(16.45953, 7.54047)$, $(16.45953, 8.45953)$, $(15.54047, 8.45953)$. All have positive camera depth $1 - 0.0425 = 0.9575$ m. This tiny square illustrates why the 32×16 example is not a practical tracking configuration.

7 Projectors: Fixed Lights and Gimballed Projection Lanterns

7.1 The projector is an inverse camera, not a camera sensor

For a world surface point, decide which projector pixel should illuminate it:

$$\boldsymbol{q}_{P_j}(t) = \pi_{P_j}\left(T_{P_j \leftarrow W}(t)\bar{\boldsymbol{x}}_W(t)\right). \tag{27}$$

In this and subsequent shorthand, π takes the first three components of the transformed homogeneous point. Use only points in front of the lens and inside the output image. Clip triangles crossing the near plane; test the nearest surface along each projector ray with a depth buffer or mesh intersection.

A projector cannot see printed markers. To calibrate it:

1. Calibrate a camera and determine its pose relative to a known surface, FF, or independently measured 3D model.
2. Project identifiable patterns at known *projector pixels*. Use Gray codes, indexed dots, or distinguishable projected markers.
3. Observe the lit pixels in the camera; invert camera distortion and intersect their rays with known surfaces, use registered depth, or triangulate with synchronized calibrated cameras.
4. Pair those recovered 3D points with their known projector pixels. With known projector intrinsics, solve projector PnP. For unknown lens parameters, gather diverse geometry/views and calibrate them separately.
5. Validate on different points, depths, and image regions. A single plane gives a homography, not a fully constrained arbitrary projector lens/distortion calibration.

Do not derive unknown surfaces from the same unverified projector model and then call them independent validation. Isolate multiple projectors' patterns by time or coding; otherwise a camera may match the wrong emitter.

7.2 The lantern has two moving parts, not one rigid pose

One handle, one articulated neck, one optical platform

The lantern's outer housing holds the handle, compute, power bank and gimbal base. A common gimbal carries the camera and projector together on a rigid optical platform. You can move the handle while that platform holds its aim, or turn the platform while the handle stays still. The camera and projector keep their mutual offset; neither keeps a fixed offset to the handle while the gimbal turns.

Let H name the lantern *housing* frame, and B the gimballed *optical carrier* frame. Throughout the lantern model, B refers to this optical carrier, not the housing. Let $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_r)$ be the actual measured angles of r gimbal joints, in radians, after encoder zero-offset correction. A two- or three-axis gimbal is allowed; its joint order, axes, pivots, signs and angle limits are calibration data.

The fixed mounting transforms are $T_{B\leftarrow C}$ and $T_{B\leftarrow P}$. The articulated transform $T_{H\leftarrow B}(\boldsymbol{\alpha})$ changes with the joints. The world camera and projector poses follow the complete chain:

$$T_{W\leftarrow B}(t) = T_{W\leftarrow H}(t)T_{H\leftarrow B}(\boldsymbol{\alpha}(t)), \tag{28}$$

$$T_{W\leftarrow C}(t) = T_{W\leftarrow B}(t)T_{B\leftarrow C}, \tag{29}$$

$$T_{W\leftarrow P}(t) = T_{W\leftarrow B}(t)T_{B\leftarrow P}, \qquad T_{P\leftarrow C} = T_{B\leftarrow P}^{-1}T_{B\leftarrow C}. \tag{30}$$

Thus the camera-to-projector transform is constant even though the camera-to-housing and projector-to-housing transforms are not. For a parked lantern, its housing may be static while its projector is still moving. Only a fixed housing *and locked joints* make the whole lantern equivalent to a static projector. Figure 2 draws these links as a coordinate transform graph.

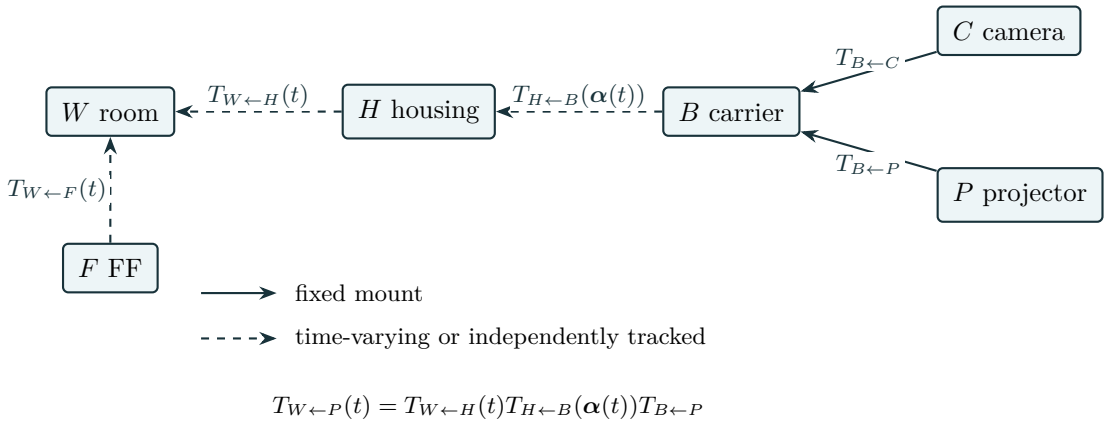


Figure 2: Arrows map point coordinates from the tail frame into the head frame; they are not motion or network arrows. Read a projector-to-world chain from right to left. The camera and projector share fixed carrier mounts, while the housing–carrier link changes as the joints turn.

7.3 Two valid ways to know where the light is

Route A: track the optical carrier directly. The camera rides on the carrier and sees the FF. With an independently known current FF world pose,

$$T_{W\leftarrow B}(t) = T_{W\leftarrow F}(t)T_{C\leftarrow F}(t)^{-1}T_{B\leftarrow C}^{-1}. \tag{31}$$

This gives the optical pose *without gimbal encoders*. Encoders remain useful for control, limits and diagnostics, but they are not required in this geometric path while optical tracking is valid. Knowing the camera's pose does not by itself locate the handle unless the housing–carrier joint relation is also known.

Route B: track the housing and measure the articulation. An independently world-aligned tracker supplies $T_{W\leftarrow H}(t)$. Synchronized measured joint angles supply $T_{H\leftarrow B}(\boldsymbol{\alpha}(t))$. Compose the chain in Equation (30). A motor command, a "stabilized" indicator, or a housing IMU alone is *not* this measurement. Motor lag, backlash, joint deflection and encoder errors must enter the uncertainty budget. When neither route is valid, bound prediction and then fade/blank output.

For object-locked projection, the room can cancel completely:

$$T_{P\leftarrow F}(t) = T_{P\leftarrow C}T_{C\leftarrow F}(t), \qquad \boldsymbol{q}_P(t) = \pi_P\big(T_{P\leftarrow F}(t)\bar{\boldsymbol{x}}_F\big). \tag{32}$$

The common gimbal preserves this direct route because it carries both devices rigidly together. It does not restore room-lock when the FF and all independent room references are lost.

7.4 A measured gimbal model, including its lever arms

If the installation uses Route B, calibrate the joint model rather than assuming every rotation occurs at the projector lens. Let $\boldsymbol{a}_j$ be joint j's unit axis expressed in that joint's fixed local frame. Its pure revolute transform is

$$\mathcal{R}_j(\alpha_j) = \begin{bmatrix} \mathrm{Exp}(\boldsymbol{a}_j\alpha_j) & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \tag{33}$$

Here Exp maps an axis–angle rotation vector to its rotation matrix using Rodrigues' formula; the timing section uses the same notation. Let A_0 map the first joint's fixed frame into H. For $1 \le j < r$, let A_j map the next joint's fixed frame into joint j's moving frame; let A_r map carrier B into the last moving frame. These constant 4×4 link transforms include pivot spacing and offsets. Then

$$T_{H\leftarrow B}(\boldsymbol{\alpha}) = A_0\mathcal{R}_1(\alpha_1)A_1\mathcal{R}_2(\alpha_2)A_2 \cdots \mathcal{R}_r(\alpha_r)A_r. \tag{34}$$

For a one-axis device the product is simply $A_0\mathcal{R}_1(\alpha_1)A_1$. Use encoder angles corrected by measured zeros, not commanded angles. Validate the model over the actual angle range and from both motion directions to reveal hysteresis/backlash.

The carrier origin need not lie on a pivot. Writing $T_{W\leftarrow H} = (R_{WH}, \boldsymbol{c}_H)$ and $T_{H\leftarrow B} = (R_{HB}, \boldsymbol{t}_{HB})$ gives

$$R_{WB} = R_{WH}R_{HB}, \qquad \boldsymbol{c}_B = \boldsymbol{c}_H + R_{WH}\boldsymbol{t}_{HB}(\boldsymbol{\alpha}). \tag{35}$$

The subscripts here abbreviate the same arrows: $R_{WH} = R_{W\leftarrow H}$, and similarly for R_{HB}, R_{WB}. Rotating a joint can therefore change *position as well as pointing*. Do not model a gimbal as orientation-only if the optics have nonzero lever arms.

If camera and projector use separate gimbals instead. This is not the baseline lantern. In that alternative, $T_{B\leftarrow C}$ and $T_{B\leftarrow P}$ cannot both be constants for one common carrier. Measure their separate housing links $T_{H\leftarrow C}(t)$ and $T_{H\leftarrow P}(t)$, and use $T_{P\leftarrow C}(t) = T_{H\leftarrow P}(t)^{-1}T_{H\leftarrow C}(t)$. The common-carrier direct shortcut must not be reused with a frozen offset.

7.5 Bench calibration, gimbal control and sensing are different jobs

First lock the gimbal and calibrate the camera/projector lenses and their rigid carrier offsets. In several stationary bench poses k, measure both devices relative to the same surveyed target. Each pair gives

$$X_k = T_{P\leftarrow F,k}T_{C\leftarrow F,k}^{-1} \simeq T_{P\leftarrow C}. \tag{36}$$

Fit one rigid transform across these observations with robust residuals on $SE(3)$, not an elementwise matrix average. A camera looking only at printed tags provides no projector pose; projected-pixel correspondences are still required. Then unlock and sweep the gimbal. The estimated camera–projector offset should stay constant while the housing–carrier relation changes.

Stabilizing the lantern is not calibrating the artwork

The gimbal's controller keeps the platform steady or follows an aim command. Spatial tracking determines where the platform actually is. Rendering chooses pixels that hit the intended surface. These are three cooperating loops, not three names for the same loop. A gimbal cannot cancel an unknown handle translation or remove parallax.

If an IMU is added, put it on the optical carrier for direct carrier prediction, calibrate its fixed camera offset, and use the carrier as the inertial body in Section 10. An IMU only in the housing does not measure the carrier's angular motion when the joints move. Using it requires the joint kinematics and rates with their timestamps; do not feed housing measurements into a carrier-state

filter as though the assembly were rigid. No IMU is required for the first visually tracked lantern prototype.

Measure camera exposure, encoder acquisition, controller telemetry and display timing in one clock convention. For housing-based prediction at intended light-emission time t_ℓ, predict *both* factors before composing:

$$\hat{T_{W\leftarrow P}}(t_\ell) = \hat{T_{W\leftarrow H}}(t_\ell) T_{H\leftarrow B}(\hat{\boldsymbol{\alpha}}(t_\ell)) T_{B\leftarrow P}. \tag{37}$$

For example, a bounded constant-joint-rate estimate is $\hat{\boldsymbol{\alpha}}(t_\ell) = \boldsymbol{\alpha}_k + \dot{\boldsymbol{\alpha}}_k(t_\ell - t_k)$, where $\dot{\boldsymbol{\alpha}}_k$ is measured joint rate at time t_k. Respect calibrated angle limits, actuator speed limits and uncertainty; this is an estimate, not a promise of actuator response. For direct carrier tracking, predict the carrier pose instead and do not apply the same joint motion twice.

During homing, transport-lock release, mode changes, joint saturation, cable snags or controller faults, blank/restrict projection according to the tested operating policy. On recovery, validate the optical pose before resuming. Keep enough tracking-camera view of known references throughout the intended aim range; pointing the projector at the art must not silently make its co-mounted camera lose every localization landmark.

7.6 Example 3: Baseline and gimbal motion both matter

Keep the overhead camera of Example 2 at its initial pose. Put a parallel projector 0.060 m to the camera's right, so $T_{C\leftarrow P}$ has identity rotation and translation $(0.060, 0, 0)$. For the world point $(0.100, 0, 0)$ and the tiny K:

$$\boldsymbol{x}_C = (0.100, 0, 1)^\top, \qquad \boldsymbol{q}_C = (17.6, 8)^\top, \tag{38}$$

$$\boldsymbol{x}_P = (0.040, 0, 1)^\top, \qquad \boldsymbol{q}_P = (16.64, 8)^\top. \tag{39}$$

Copying the camera pixel directly into the projector is wrong even with parallel optical axes.

Now keep the housing fixed with its axes aligned to W and its origin at a pivot at $(0, 0, 1)$ m in W. Consider a simple one-axis gimbal. At zero angle the carrier looks downward, with $R_0 = \mathrm{diag}(1, -1, -1)$. Let $R_{H\leftarrow B}(\alpha) = R_y(\alpha) R_0$, where $R_y(\alpha)$ is a right-handed rotation about the housing's $+y$ axis:

$$R_y(\alpha) = \begin{bmatrix} \cos\alpha & 0 & \sin\alpha \\ 0 & 1 & 0 \\ -\sin\alpha & 0 & \cos\alpha \end{bmatrix}. \tag{40}$$

Place the camera at the pivot and the projector at carrier offset $(0.060, 0, 0)$ m. At $\alpha = 30°$, the projector's world center and forward direction become

$$\boldsymbol{c}_{W,P} = \begin{bmatrix} 0.060\cos\alpha \\ 0 \\ 1 - 0.060\sin\alpha \end{bmatrix} \simeq \begin{bmatrix} 0.05196 \\ 0 \\ 0.97000 \end{bmatrix} \text{ m}, \qquad \boldsymbol{p}_{W,P} = \begin{bmatrix} -\sin\alpha \\ 0 \\ -\cos\alpha \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0 \\ -0.86603 \end{bmatrix}. \tag{41}$$

The handle has not moved, but the projector has both tilted and shifted 30 mm vertically. Treating the housing pose as the projector pose would miss both effects.

8 Surfaces, Depth, Rays, and the Limits of a Canvas

8.1 A pixel becomes a ray

First invert the distortion model for a measured pixel, for example with `undistortPoints`, to obtain normalized coordinates (x_n, y_n). The local ray is $\boldsymbol{d}_V = (x_n, y_n, 1)^\top$. With world-to-device

rotation $R = R_{V\leftarrow W}$ and translation $\boldsymbol{t} = \boldsymbol{t}_{V\leftarrow W}$, set

$$\boldsymbol{o}_W = -R^\top \boldsymbol{t}, \qquad \boldsymbol{d}_W = R^\top \boldsymbol{d}_V, \qquad \boldsymbol{x}_W(\lambda) = \boldsymbol{o}_W + \lambda \boldsymbol{d}_W. \tag{42}$$

The direction need not be unit length; therefore λ is not generally Euclidean distance.

A plane is defined by unit normal $\boldsymbol{n}$ and scalar offset β: $\boldsymbol{n}^\top \boldsymbol{x}_W + \beta = 0$. Substitution gives

$$\lambda = -\frac{\boldsymbol{n}^\top \boldsymbol{o}_W + \beta}{\boldsymbol{n}^\top \boldsymbol{d}_W}. \tag{43}$$

Reject near-zero denominators, $\lambda \le 0$, points outside the finite surface, and intersections hidden behind a nearer surface. The table plane is $\boldsymbol{n} = (0, 0, 1)^\top, \beta = 0$. Raycasting is the same operation for a mouse, a tracked phone, or a projector; their ray origins and calibrations differ. Figure 3 sketches one shared surface point.

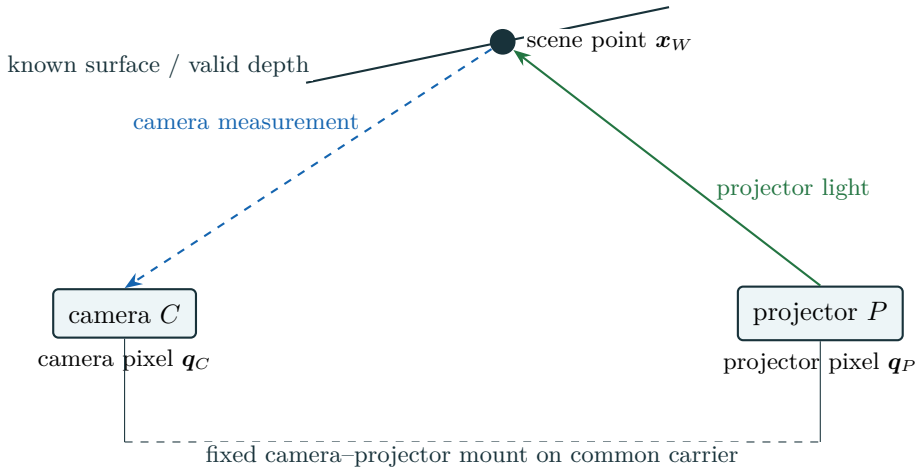


Figure 3: A camera pixel identifies a ray, not a unique 3D point. A known surface or valid depth supplies the missing location. The projector must illuminate that same point from its different lens center, so camera and projector pixels generally differ. This sketch is not to scale.

8.2 Depth is an additional measurement, not a replacement for pose

If a depth image reports *axial depth* z_D in metres at an undistorted depth pixel, then

$$\boldsymbol{x}_D = z_D(x_n, y_n, 1)^\top, \qquad \bar{\boldsymbol{x}}_C = T_{C\leftarrow D}\bar{\boldsymbol{x}}_D, \qquad \bar{\boldsymbol{x}}_W = T_{W\leftarrow D}(t)\bar{\boldsymbol{x}}_D. \tag{44}$$

If the sensor instead reports radial range ℓ, use $\boldsymbol{x}_D = \ell\, \boldsymbol{d}_D/\|\boldsymbol{d}_D\|$. Depth and color have distinct intrinsics, distortion and image sizes. Use the measured color/depth transform and temporal registration; holes, multipath, reflective surfaces and occlusion remain.

Example 4. At pixel $(20, 10)$ with the tiny K, axial depth 2 m produces $\boldsymbol{x}_D = (0.5, 0.25, 2)$ m. If the reported value were radial range 2 m, the result would instead be $2(0.25, 0.125, 1)/\sqrt{1.078125}$ m.

8.3 Exactly when a homography is enough

In an undistorted pinhole model, for a plane whose *world coordinates* are $(X, Y, 0)$, let $\boldsymbol{r}_1, \boldsymbol{r}_2$ be the first two columns of $R_{P\leftarrow W}$ and $\boldsymbol{t} = \boldsymbol{t}_{P\leftarrow W}$. The planar map is

$$\bar{\boldsymbol{q}}_P \sim H_P(X, Y, 1)^\top, \qquad H_P = K_P[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}]. \tag{45}$$

The symbol $\sim$ means equality up to nonzero scale, followed by division by the third component. For two devices seeing that plane, $H_{P\leftarrow C} = H_P H_C^{-1}$, when H_C is invertible. Nonzero lens distortion needs explicit undistortion/redistortion around the homography. A table plane, a raised FF top, and its sloping faces are different planes.

For a general mesh, render from each projector's own calibrated view, using its current pose and lens warp. If the physical surface itself moves, predict its pose as well as the projector's. Anamorphosis additionally depends on a selected viewer-eye pose; a viewer camera is not automatically the projector camera.

8.4 Light overlap is radiometry, not just geometry

At a surface point, let g_j be projector j's measured linear irradiance-per-command gain (including distance, angle and reflectance), and $a_j \in [0, 1]$ its linear-light command. With ambient contribution E_{amb},

$$E = E_{\text{amb}} + \sum_j g_j a_j. \tag{46}$$

For desired added light $E_\star$ and nonnegative spatial weights w_j summing to one over available projectors, one simple allocation is $a_j = \text{clip}(w_j E_\star / g_j, 0, 1)$ for $g_j > 0$. Clipping means the target may be unattainable. Apply measured transfer functions for display gamma/color; black levels cannot cancel ambient light. A blanket "each node uses $1/n$ brightness" works only for equal calibrated gains and actual overlapping coverage. Modifying LED current requires a supported hardware interface, not an assumed HDMI feature.

Smoke/haze permits visible beams, not arbitrary opaque 3D surfaces. Keep the beam-sculpture mode separate from the surface-mapping mode and use venue-approved haze, ventilation and accessible alternatives.

9 Time: Calibrate When the Light Lands, Not When a Packet Arrives

There are two kinds of lateness

A frame can be geometrically accurate for a carrier pose from 60 ms ago. That does not make it accurate for the moving carrier now. Making a renderer run smoothly addresses cadence; measuring and predicting to the light-emission time addresses age. These are different problems.

9.1 Timestamps and clock alignment

For one frame, define capture/exposure midpoint t_c, tracking completion t_s, render sampling time t_r, and intended light-emission time t_ℓ. End-to-end age is

$$\tau = t_\ell - t_c = \tau_{\text{readout}} + \tau_{\text{queue}} + \tau_{\text{solve}} + \tau_{\text{transport}} + \tau_{\text{render}} + \tau_{\text{present/display}}. \tag{47}$$

These terms must be measured without double-counting overlapping stages. Include exposure integration, rolling shutter and display scanout in the timing model where relevant. A global shutter avoids row-dependent exposure poses but not exposure blur or downstream buffering. Application-submit time is not camera exposure time.

Device monotonic clock t_d maps approximately to session clock t_W by

$$t_W = a_d t_d + b_d. \tag{48}$$

Here a_d is clock rate/skew and b_d offset; estimate them over time and publish uncertainty. Arrival time includes network delay. As a simple offset probe, client send/receive times t_1, t_4 and server receive/send times t_2, t_3 give $\hat{b} = ((t_2 - t_1) + (t_3 - t_4))/2$ and round-trip network estimate $(t_4 - t_1) - (t_3 - t_2)$, *assuming approximately symmetric delays and negligible drift during the probe.* Asymmetry creates offset error. Do not subtract different computers' raw monotonic clocks.

9.2 Interpolation is behind; prediction is ahead

For known body-to-world poses at $t_0 < t_1$ bracketing a desired time t, write their centers $\boldsymbol{c}_0, \boldsymbol{c}_1$ and rotations R_0, R_1. Let $\alpha = (t - t_0)/(t_1 - t_0)$, with $0 \le \alpha \le 1$:

$$\boldsymbol{c}(t) = (1 - \alpha)\boldsymbol{c}_0 + \alpha \boldsymbol{c}_1, \qquad R(t) = R_0 \operatorname{Exp}\Big(\alpha \operatorname{Log}(R_0^\top R_1)\Big). \tag{49}$$

Exp maps a three-component rotation vector to a rotation matrix (Rodrigues); Log maps a relative rotation to its principal rotation vector. Quaternion SLERP is another way to perform this orientation interpolation with appropriate sign/shortest-path handling. Do not call SLERP directly on a 4×4 matrix.

For display time *after* the latest measurement, use a model. With world linear velocity $\boldsymbol{v}$, body angular velocity $\boldsymbol{\omega}$ and horizon $h = t_\ell - t_k \ge 0$,

$$\hat{\boldsymbol{c}}(t_\ell) = \hat{\boldsymbol{c}}_k + \hat{\boldsymbol{v}}_k h, \qquad \hat{R}(t_\ell) = \hat{R}_k \operatorname{Exp}(\hat{\boldsymbol{\omega}}_k h). \tag{50}$$

This is constant-velocity extrapolation, not a guarantee. Cap horizon and uncertainty; it cannot anticipate an unexpected impact or carrier slew. Measure velocity in the stated frame, not by differencing unrelated world-to-camera translation vectors.

If the surface is FF-attached and both surface and projector move, evaluate

$$T_{P \leftarrow F}(t_\ell) = T_{W \leftarrow P}(t_\ell)^{-1} T_{W \leftarrow F}(t_\ell). \tag{51}$$

Predicting only one side still produces a temporal mismatch. An optional neural predictor must earn its place against these baselines on held-out motion, report uncertainty, and fall back safely. Neither an NPU nor adaptive process noise creates missing geometric evidence. Figure 4 shows the two timelines involved.

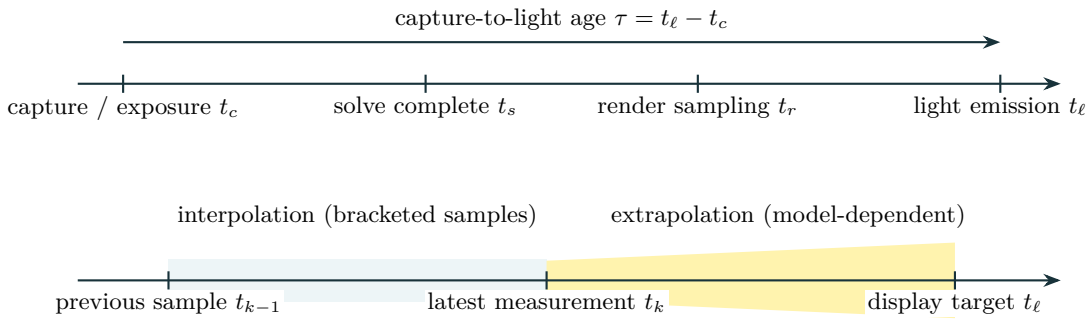


Figure 4: Smooth frame cadence does not remove the age of the observations. Interpolation uses bracketing samples; beyond the latest measurement, display-time pose is a prediction with uncertainty. Camera, joint and display timing must use a compatible clock. Timeline spacing and uncertainty shading are schematic, not measurements.

9.3 Example 5: Smooth does not mean aligned

A lantern's optical carrier slews at 90°/s with 60 ms capture-to-light age; the gimbal can rotate the carrier even while the handle stays still. It turns 5.4° during that delay. At a 2 m perpendicular surface, the uncompensated angular displacement is about $2\tan(5.4°) = 0.1891$ m. Even a 1 ms clock error at that speed contributes approximately 3.14 mm at 2 m. This is why "millimetre precision" must state motion and timing.

10 Tracking, IMUs, Dropouts, and Recovery

10.1 A visual-only filter and a visual–inertial filter are not the same

The repository's pose EKF smooths visual poses with estimated linear and angular velocity. That is useful; it is not evidence of IMU fusion. For a proposed body IMU, a suitable nominal state is

$$\boldsymbol{x} = (\boldsymbol{c}, \boldsymbol{v}, R, \boldsymbol{b}_a, \boldsymbol{b}_g). \tag{52}$$

Here $\boldsymbol{c}$ is body origin in W, $\boldsymbol{v}$ world velocity, $R = R_{W\leftarrow B}$, and $\boldsymbol{b}_a, \boldsymbol{b}_g$ are three-component accelerometer and gyroscope biases. Align the IMU axes and lever arm to B first. For simplicity this model places the body origin at the IMU; use the fixed camera/IMU offset when converting a visual observation to this origin.

Let measured specific force be $\boldsymbol{a}_m$, measured angular rate $\boldsymbol{\omega}_m$, and gravity $\boldsymbol{g} = (0, 0, -9.81)^\top$ m/s^2 in a z-up room. Over a short step Δt,

$$\boldsymbol{a}_W = R(\boldsymbol{a}_m - \boldsymbol{b}_a) + \boldsymbol{g}, \tag{53}$$

$$\boldsymbol{c}^- = \boldsymbol{c} + \boldsymbol{v}\Delta t + \tfrac{1}{2}\boldsymbol{a}_W \Delta t^2, \tag{54}$$

$$\boldsymbol{v}^- = \boldsymbol{v} + \boldsymbol{a}_W \Delta t, \tag{55}$$

$$R^- = R\,\mathrm{Exp}((\boldsymbol{\omega}_m - \boldsymbol{b}_g)\Delta t). \tag{56}$$

Biases are approximately constant over this step with random-walk noise. At rest, specific force balances gravity after rotation. Accelerometer bias accumulates position error; a magnetometer near stage metal/electronics is not a reliable absolute yaw guarantee. An IMU does not maintain drift-free position through arbitrary occlusion.

10.2 The uncertainty update, with rotations treated correctly

Use a 15-component local error $\delta\boldsymbol{x} = (\delta\boldsymbol{c}, \delta\boldsymbol{v}, \delta\boldsymbol{\theta}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g)$. Orientation error is defined on the right: $R_{\text{true}} \simeq R\,\mathrm{Exp}(\delta\boldsymbol{\theta})$. Write corrected IMU samples as $\boldsymbol{a} = \boldsymbol{a}_m - \boldsymbol{b}_a$, $\boldsymbol{\omega} = \boldsymbol{\omega}_m - \boldsymbol{b}_g$. The first-order continuous error Jacobian F_c is

$$F_c = \begin{bmatrix} 0 & I & 0 & 0 & 0 \\ 0 & 0 & -R[\boldsymbol{a}]_\times & -R & 0 \\ 0 & 0 & -[\boldsymbol{\omega}]_\times & 0 & -I \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad F \simeq I + F_c\Delta t, \quad P^- = FPF^\top + Q_d. \tag{57}$$

Each block is 3×3; Q_d is discrete process-noise covariance, obtained by integrating rotated accelerometer/gyro noise and bias random walks over the step. Tune from sensor characterization, not visual preference; use finer integration for rapid motion.

For frozen Jacobians over a short step, let Q_c be the 12×12 continuous spectral-density matrix of independent accelerometer noise, gyro noise, accelerometer-bias walk and gyro-bias walk (three components each). Their error-state injection matrix and discretization are

$$\mathcal{G} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -R & 0 & 0 & 0 \\ 0 & -I & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & I \end{bmatrix}, \qquad Q_d = \int_0^{\Delta t} \Phi(s)\mathcal{G}Q_c\mathcal{G}^\top\Phi(s)^\top\, ds, \quad \Phi(s) = e^{F_c s}. \tag{58}$$

Each block in $\mathcal{G}$ is 3×3; Q_c must use consistent SI units. A first-order approximation is $Q_d \simeq \mathcal{G}Q_c\mathcal{G}^\top\Delta t$ when sufficiently small time steps justify it.

Convert visual PnP through Equations (22) and (31) into a body pose measurement $(\boldsymbol{c}_z, R_z)$ *at exposure time*. Transform its covariance into the same world-position/body-angle convention. Let N be its 6×6 covariance including relevant anchor uncertainty. The innovation $\boldsymbol{r}$, observation Jacobian H, innovation covariance S, and gain L are

$$\boldsymbol{r} = \begin{bmatrix} \boldsymbol{c}_z - \boldsymbol{c}^- \\ \mathrm{Log}((R^-)^\top R_z) \end{bmatrix}, \qquad H = \begin{bmatrix} I & 0 & 0 & 0 & 0 \\ 0 & 0 & I & 0 & 0 \end{bmatrix}, \tag{59}$$

$$S = HP^-H^\top + N, \qquad L = P^-H^\top S^{-1}, \qquad \delta\hat{\boldsymbol{x}} = L\boldsymbol{r}. \tag{60}$$

Gate outliers using $\boldsymbol{r}^\top S^{-1}\boldsymbol{r}$ and independent geometric checks; a probability-based threshold requires a justified noise model. Apply position, velocity and bias corrections additively; apply $R^+ = R^- \mathrm{Exp}(\delta\hat{\boldsymbol{\theta}})$. Before resetting the local error to zero, use the Joseph covariance update and first-order orientation reset Jacobian:

$$\widetilde{P} = (I - LH)P^-(I - LH)^\top + LNL^\top, \tag{61}$$

$$J_{\mathrm{reset}} = \mathrm{diag}(I, I, I - \tfrac{1}{2}[\delta\hat{\boldsymbol{\theta}}]_\times, I, I), \qquad P^+ = J_{\mathrm{reset}}\widetilde{P}J_{\mathrm{reset}}^\top. \tag{62}$$

This derivation is a first-order local error-state model, not a ready-made global VIO implementation. Large inconsistent rotations should trigger validated relocalization rather than a tiny-angle correction.

For a delayed visual result, correct a stored state at its exposure time and replay the intervening IMU samples. Applying an old image as if captured now produces a biased state. Predicted/filtered samples from the same measurements are correlated: do not fuse them as independent sensors.

10.3 An explicit live state machine

State	Evidence	Rendering policy
Unregistered	No valid world alignment	Setup/status only; no room-locked light
Tracking	Fresh accepted pose and valid calibration	Render at estimated display time
Weak	Small/planar marker, poor coverage, uncertain map	Broader/less sensitive visuals; show quality
Coasting	No new accepted fix; bounded model prediction	Short, uncertainty-limited continuation
Lost	Age, uncertainty or motion budget exceeded	Fade/blank spatial output; preserve artwork data
Relocalizing	New plausible absolute observations	Validate before accepting a new world pose
Relative-only	Relative tracking survives, world link lost	Optional explicitly labelled object-locked mode

A known static projector can keep rendering room-locked artwork during a camera dropout if its pose and the surface remain trusted. A moving projector must not simply freeze its last image and call that world-lock. For a carried lantern, rapid carrier or handle motion or uncertain pose should cause earlier blanking. These are alignment policies, not optical certification. Figure 5 summarizes the recovery loop.

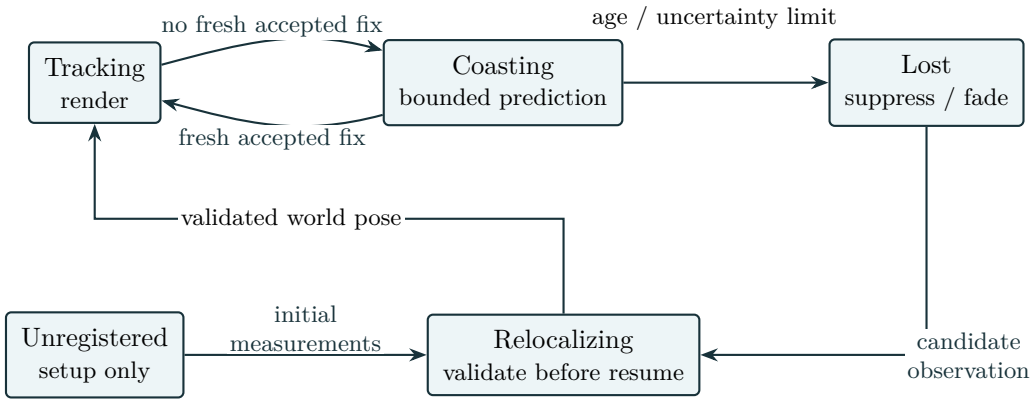


Figure 5: Simplified recovery loop: losing a pose does not delete the canvas. The full state table also covers Weak and Relative-only operation; neither creates missing room-reference evidence. Homing or controller faults can independently suppress projection even when a geometric pose is available.

Recovery after a long loss. Detect full-frame; solve multiple candidates where appropriate; check positive depth and model consistency; validate against fixed landmarks or an independent tracker; require a stable sequence according to the installation's tested acceptance policy. Reset or reinitialize the estimator if the old prediction is inconsistent. Keep canvas coordinates fixed. A cosmetic fade can hide a pose correction, but smoothing must not make a known wrong calibration appear valid.

Bumps. Compare observed fixed-landmark corners with predictions from the stored fixed-camera pose. A fresh PnP fit may have a low residual after a bump, so monitoring only its post-fit residual misses the event. Separate camera displacement, FF motion, bad correspondences, and lens changes using independent constraints. Update only affected transforms.

11 Collaborative Canvas and the Spatial Pipe

Share the score, not everyone's screen recording

Transmit strokes, attachments, transforms and show events. Each capable node renders locally from its own current pose. This keeps carrier motion off the Wi-Fi critical path. The network still matters for agreement, ordering, authentication and shared calibration; it does not replace local sensing.

11.1 Choose a practical first topology

Use one session coordinator for identity, authoritative calibration revisions, content event order, and discovery. Keep tracking and projection local. On coordinator loss, already registered nodes may continue trusted local rendering and buffer artwork events, but cannot invent conflicting new world epochs. A router is not necessarily an application server.

This is **distributed rendering with coordinated session state**, not a claim of fully decentralized consensus. MQTT normally involves a broker; WebSockets supply transport, not discovery or peer consensus. A later peer system must define leader election, partitions and merge rules explicitly.

11.2 Artwork must declare what it is attached to

For a stroke vertex $\bar{\boldsymbol{x}}_A$ stored in attachment frame A,

$$\bar{\boldsymbol{x}}_W(t) = T_{W\leftarrow A}(t)\bar{\boldsymbol{x}}_A, \qquad \boldsymbol{q}_P(t) = \pi_P(T_{P\leftarrow W}(t)T_{W\leftarrow A}(t)\bar{\boldsymbol{x}}_A)\,. \tag{63}$$

Choose $A = W$ for room-lock, $A = F$ for FF-lock, or a tracked object. Store samples at their creation time and in their declared attachment, not in the current producer-camera frame.

A stroke created in F at t_0 and then baked into the room uses $\bar{\boldsymbol{x}}_W = T_{W\leftarrow F}(t_0)\bar{\boldsymbol{x}}_F$ forever. A genuinely FF-attached stroke instead uses $T_{W\leftarrow F}(t)$ every frame. This one decision determines whether the artwork follows the prop. Figure 6 shows both behaviors on one ruler.

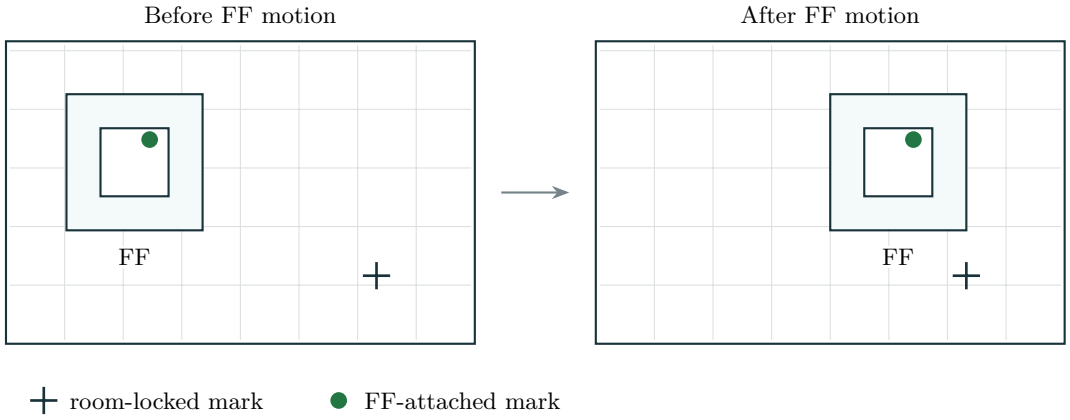


Figure 6: Room-locked marks stay fixed when the FF moves. FF-attached marks keep their local coordinates and move with it. Moving a lantern changes the rendering viewpoint, not either mark's attachment. The two panels use the same room ruler.

If a deliberate coordinate rebase creates a new room epoch W' with $G = T_{W'\leftarrow W}$, transform *all* room-locked geometry and world poses consistently, e.g. $\bar{\boldsymbol{x}}_{W'} = G\bar{\boldsymbol{x}}_W$ and $T_{W'\leftarrow B} = GT_{W\leftarrow B}$.

Do not overwrite the origin silently while leaving old strokes untouched. A map correction and a coordinate rebase are different operations.

11.3 A concrete, proposed message contract

The following is a **new protocol specification**, not the current `fulcrum_poses.json` schema. Matrix rows are row-major serialized arrays but multiply column vectors as in Equation (1). The pose covariance uses order $(\delta c_x, \delta c_y, \delta c_z, \delta\theta_x, \delta\theta_y, \delta\theta_z)$, world translation/right-local rotation convention.

```
{
  "schema": "shared-light.pose/1",
  "session_id": "gallery-session",
  "world_epoch": 3,
  "calibration_revision": "rig-r17",
  "geometry_revision": "ff-v1-survey-02",
  "device_id": "lantern-a",
  "sequence": 1842,
  "from_frame": "optical-carrier:lantern-a",
  "to_frame": "world:3",
  "units": "m",
  "clock_id": "session-monotonic",
  "capture_time_s": 12.400,
  "estimate_time_s": 12.400,
  "valid_until_s": 12.430,
  "clock_sigma_s": 0.001,
  "T_to_from": [[1,0,0,0.4], [0,1,0,0.2],
                [0,0,1,1.5], [0,0,0,1]],
  "status": "tracking",
  "evidence_ids": ["cam-a:frame-442"],
  "covariance_order": ["cx","cy","cz","rx","ry","rz"],
  "covariance_diagonal": [0.000025,0.000025,0.0001,
                          0.0001,0.0001,0.0004]
}
```

Numbers are illustrative, not measurements. The example only serializes a diagonal covariance; a production full matrix must explicitly declare its layout. A predicted estimate must have its own estimate time and keep the original capture time. `valid_until` is a policy budget, not proof of future accuracy. Track measurement provenance to avoid double-counting the same observation across relays.

Use separate message families:

- **Calibration snapshots:** reliable, retained, versioned and acknowledged; intrinsics, image processing, rigid offsets, marker map, clock conventions and validation report.
- **Pose telemetry:** latest valid state wins; sequence checks, timestamps, expiry, uncertainty, evidence provenance.
- **Artwork events:** reliable begin/append/end operations with stroke ID, author, attachment, sample times, local sequence and style.
- **Show commands:** authenticated mode changes scheduled in session time, with acknowledgements and an expiry/late policy.

For a first version, each stroke has one writer; append indices make retries idempotent. A session event order resolves deletes and style changes; tombstones prevent deleted strokes reappearing after reconnect. Buffer offline edits with unique IDs and reconcile explicitly. Pose loss must not delete the shared canvas.

11.4 Queues, integration and access control

Bound queues and discard superseded camera frames rather than process an ever-older FIFO. Keep exposure timestamps through capture, tracking and rendering. A larger queue can improve throughput statistics while worsening interactive alignment.

Use worker processes/native code for heavy vision; Python threads alone do not guarantee CPU parallelism. Keep TouchDesigner operator access on its supported main-thread path. Exchange bounded immutable pose/image snapshots; shared memory may still involve copies. Network OSC is not zero-copy memory. Monitor tracking cadence, render cadence, GPU deadlines, data age, queue depth and rejected measurements separately.

The existing TD bridge can execute Python remotely. Keep it on localhost or a trusted restricted network; do not expose it as a public audience API. Use a small authenticated allow-listed control API for phones, role-based permissions for calibration changes, and explicit consent for camera capture. Do not broadcast raw video unless it is specifically needed and agreed.

12 Initial and On-the-Go Calibration: Operational Recipes

12.1 Initial survey: before the first performance

Figure 7 summarizes the order.

1. [HW] **Make it mechanically and optically safe.** Use a rigid optical carrier on its gimbal mount, strain relief, ventilation, tested power rails, conservative optical output and an accessible stop. Bench-test before any handheld use.

2. [I/O] **Inventory the exact streams.** Record lens, serial ID, image size, crop, stabilization, exposure timing, depth units and output mode. Lock changing optics/warps.

3. [I/O] **Calibrate lenses and timing.** Use diverse calibration views and held-out tests. Fit depth/color registration, IMU alignment and clock offsets where those sensors exist.

4. [HW] **Survey the FF and static references.** Measure actual corners, reserve unique IDs, version the geometry, and provide fixed room references if the FF is allowed to move.

5. [I/O] **Establish the room once.** Hold the FF fixed; choose $T_{W\leftarrow F}(t_0)$; solve camera poses with Equation (19); verify static-camera baselines.

6. [I/O] **Calibrate every projector.** Capture coded patterns and physical surface points. Solve projector lens/pose parameters and rigid camera–projector offsets. For a lantern, fit the camera and projector on the shared carrier as one rigid pair.

7. [I/O] **Unlock, sweep and validate the gimbal.** With the joints locked, complete the rigid lens and offset fits above. Then unlock and sweep the joints: the measured camera–projector offset must stay invariant while the housing–carrier relation changes. The direct Route A prototype needs no encoder. Before enabling Route B, calibrate the joint axes, pivots, signs and encoder zeros, and align the camera, encoder and tracker clocks. Validate hold/follow/home/transport-lock transitions and controller-fault suppression before the show.

8. [VIS] **Measure independent alignment.** Use test points not used for fitting, including multiple depths and image edges. Test moving devices, dropout, and camera bumps.

9. [API] **Publish one coherent snapshot.** Stage all device results, validate them, atomically activate one revision, and require node acknowledgement. Retain the last known-good revision and report partial failure without mixing old and new results.

10. [APP] **Rehearse the performance state machine.** Show tracking/weak/lost states, confirm which artwork is room-locked or object-locked, rehearse blanking and recovery, then enable the show.

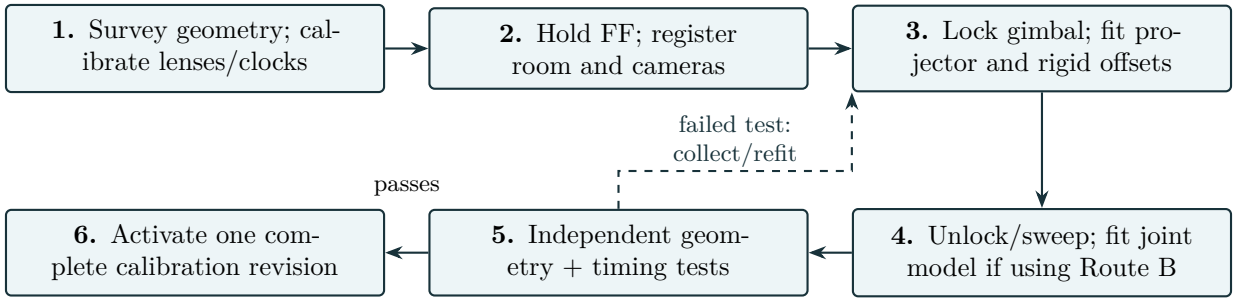


Figure 7: Initial calibration produces a validated snapshot, not a permanent promise. Revisit affected fits after optical or mechanical changes. Routine lantern motion updates pose; it does not require a new lens calibration each frame.

12.2 During a show: what to recalibrate, and when

Event	Correct operation	Do not do this
FF moves, fixed reference remains	Update $T_{W \leftarrow F}(t)$; update FF-attached content and device localization	Reset W on every FF observation
Lantern carrier or housing moves	Track the optical carrier (directly, or housing plus measured joints); predict to display time	Refit lens distortion every frame
Static camera is bumped	Exclude its old world link; relocalize to fixed landmarks or independently tracked FF	Infer camera motion from post-fit FF residual alone
Static projector is bumped	Blank it; track its attached body or rerun projected correspondences	Assume a separate static camera tracks its motion
Marker temporarily hidden	Bound prediction and uncertainty; fade as needed	Keep calling stale pose "tracked"
Gimbal homes, reaches a limit or faults	Suppress the affected projection; check controller state and reacquire/-validate the carrier pose before resume	Treat motor commands or a homed flag as a measured pose
All room references lost	Relative mode or blackout; keep artwork stored	Promise room-lock from IMU integration indefinitely
Focus/zoom/mount changes	Recheck lens or affected rigid calibration	Treat mechanical changes as a software smoothing problem
New device joins	Load matching snapshot, calibrate its own lens and offsets, localize and validate	Copy another device's K or just scan an ID

12.3 Python commands available in this repository today

Run from the project root in a **Windows Command Prompt**. The project CV interpreter is `C:\Python314\python.exe`; the base Anaconda interpreter lacks the required CV modules. These commands are documented here, *not executed as part of writing this document*. Calibration patterns alter live projector output and require an attended, stationary setup.

```
:: Existing offline rig solver: webcam + projector + Kinect by default.
:: Requires the running TD bridge and the expected FF network.
C:\Python314\python.exe tools\fulcrum\ff_calibrate.py

:: Explicitly omit Kinect registration:
C:\Python314\python.exe tools\fulcrum\ff_calibrate.py --no-kinect

:: Inspect CLI options without starting calibration:
C:\Python314\python.exe tools\fulcrum\ff_calibrate.py --help

:: Alternative standalone tracker; owns camera capture.
:: Do not run alongside TD capture for those same devices.
C:\Python314\python.exe tools\fulcrum\fulcrum_track.py
```

The old `-kinect` flag is not accepted by the current parser. The default recalibrates the *configured webcam/projector rig and attempts Kinect registration*; it does not discover every camera on the network or calibrate arbitrary lanterns.

TD path, not the standalone path. `td_ff_live.py` is a TD module; its worker is `ff_worker.py`. Running the module as a normal standalone Python CLI is not the supported live setup. Existing TD setup scripts can build/modify the network and should only be used on the intended project. The bridge's documented script invocation form is

```
node tools/td/td.mjs --file <existing-setup-script.py>
```

The angle-bracket argument is a placeholder, not an executable command. No general-purpose `xr calibrate-all`, mobile daemon, IMU fusion service or gimbal control API is claimed to exist yet.

12.4 Repository audit: current behavior versus the proposed system

This is a source inspection snapshot, not a live hardware diagnosis.

1. **All-device publication is not transactional.** In `ff_calibrate.py`, `main()` writes `calibration_current.json` and calls `export_viewer()` before `phase_kinect()`. The export can therefore use the *previous* Kinect registration. The archive is also written before that phase, and its `validated` field describes the earlier fit, not complete all-camera success. Kinect SDK import failure returns a skip. Default participation does not prove a coherent all-camera result.

2. **The room anchor still depends on a frozen webcam pose.** `export_viewer()` and `td_ff_live.py` use `aruco_solve_frozen.json` for the table/world link. Re-solving the FF alone does not refresh that independent link if the webcam moved. This is why rerunning one command is not an unconditional stale-anchor fix.

3. **Fixed camera–projector links assume fixed hardware.** The live module constructs `projector_from_camera` from stored calibration and Kinect registration. That is appropriate for unchanged rig geometry, not independently moving devices.

4. **The webcam fit is not a full lens calibration.** The offline solver refines a constrained pinhole model with priors and zero webcam distortion. Kinect K/D is fitted to factory mapper samples, not a fresh independent optical calibration of arbitrary cameras.

5. **Worker isolation exists; a complete inertial pipe does not.** `ff_worker.py` uses a separate process and shared-memory buffers with image copies. `pose_ekf.py` is a visual pose/velocity filter; this inspected path does not implement the proposed 200 Hz IMU/NPU system. Live timestamps include application-level timing, not a demonstrated hardware exposure-to-photon synchronization system.
6. **Hold/fade is a visual policy, not guaranteed tracking.** TD defaults include a 1.5 s hold/fade window and a 0.25 s filter reacquisition setting, subject to controls. This is not a measured safe carrier prediction horizon. The separate painted-solid tracker has different constants and is not the stage-scale FF document model.

These are implementation tasks in Section 14; this document does not silently modify the live rig.

13 Joint Refinement and an Honest Validation Plan

13.1 A connected calibration graph

Think of frames as nodes and measured transforms as edges. Every device needing room-lock must have a valid, time-consistent path to W. Fix the room frame (or one surveyed anchor pose) so the optimization does not float. Metric surveyed geometry fixes scale; unconstrained monocular mapping otherwise has a scale ambiguity.

Let Θ collect the unknown lens parameters, body/device offsets, per-time poses and any explicitly estimated surface parameters. For camera observation i at time t_i of a point stored on attachment frame A_i, predict

$$\hat{\boldsymbol{q}}_i = \pi_{C_i}\big(T_{C_i \leftarrow W}(t_i) T_{W \leftarrow A_i}(t_i) \bar{\boldsymbol{x}}_{A_i,i}\big). \tag{64}$$

Projector correspondences use the same expression with P_j instead of C_i. If axial depth observation z_i is associated with an independently specified corresponding world point $\boldsymbol{x}_{W,i}$, its residual is

$$e_i^D = z_i - \boldsymbol{e}_3^\top \left(R_{D \leftarrow W}(t_i)\boldsymbol{x}_{W,i} + \boldsymbol{t}_{D \leftarrow W}(t_i)\right), \qquad \boldsymbol{e}_3 = (0,0,1)^\top. \tag{65}$$

Do not back-project a depth sample with itself and call the resulting zero depth residual an independent constraint. Surface correspondence, registration, occlusion and noise must be modeled.

For visual/projector residual vectors $\boldsymbol{e}_i$ in pixels, depth errors e_i^D in metres, corresponding covariances Σ_i and variances $\sigma_{D,i}^2$, and explicit survey/rig priors E_{prior}, a dimensionally consistent joint objective is

$$\Theta^\star = \arg\min_{\Theta} \left[\sum_{i \in \mathcal{V}} \rho(\boldsymbol{e}_i^\top \Sigma_i^{-1} \boldsymbol{e}_i) + \sum_{i \in \mathcal{D}} \rho((e_i^D)^2/\sigma_{D,i}^2) + E_{\text{prior}}(\Theta) \right]. \tag{66}$$

$\mathcal{V}$ indexes image correspondences and $\mathcal{D}$ valid depth constraints. Parameterize rotations with local increments. With stacked whitened residual vector $\boldsymbol{r}(\Theta)$ and its Jacobian J, a Levenberg–Marquardt step solves $(J^\top J + \mu I)\Delta\Theta = -J^\top \boldsymbol{r}$, where $\mu > 0$ is damping, after appropriate parameter scaling and robust reweighting. Convergence is not proof of correctness or sub-millimetre accuracy. Use this offline or in a controlled background refinement, not as an unbounded main-render-thread operation.

13.2 Measure what an audience can actually see

Define tolerances per installation, not one universal magic threshold. Report at least:

- **Static spatial accuracy:** independent projected test targets measured on physical surfaces, in mm, at several depths and viewing regions. Separate in-fit and held-out points.

- **Dynamic registration:** projected-target error while the rig moves through specified speeds/accelerations; report median, 95th percentile, maximum and failure counts with sample conditions.
- **Timing:** exposure-to-light latency and jitter, clock-offset uncertainty, tracking rate, render rate, rejected/dropped frames and pose age. Use independent timing evidence, such as a high-speed-camera or photodiode experiment with a defined stimulus.
- **Recovery:** duration and alignment error after FF occlusion, a bumped camera, moving prop, map reset, network loss and reconnect. Verify the world canvas does not jump or disappear.
- **Collaboration:** duplicate/reordered events, concurrent authors, deleted strokes, incompatible calibration revisions, unauthorized clients and coordinator restart.
- **Human/physical limits:** optical exposure assessment, enclosure temperature, runtime, mass balance, cable snag risks, accessibility and venue operating constraints.

For a near-normal planar surface at range L, small angular uncertainty σ_θ contributes approximately $L\sigma_\theta$ displacement. If independent small transverse translation uncertainty is σ_c, an illustrative error budget is $\sigma_{\text{hit}} \simeq \sqrt{\sigma_c^2 + (L\sigma_\theta)^2}$. Add correlated calibration, surface and timing effects rather than treating this approximation as an exact global bound.

13.3 Power arithmetic without marketing units

Battery energy is watt-hours (Wh), power is watts (W), and approximate runtime is

$$t_{\text{run,h}} \simeq \frac{\eta E_{\text{battery,Wh}}}{P_{\text{compute}} + P_{\text{projectors}} + P_{\text{sensors}} + P_{\text{gimbal}} + P_{\text{cooling/network}}}. \tag{67}$$

η is usable efficiency including conversion and reserve. For an illustrative 74 Wh bank, $\eta = 0.85$ and a *measured* total 40 W load including motors, runtime would be about 1.57 h; this is not a measurement of the proposed lantern. Capacity advertised in mAh needs its reference voltage.

A 100 W bank may supply only 5 V/3 A on the port used by a Pi. Verify the required profile, available current under simultaneous use, fusing, and the gimbal's idle, slew and startup peaks, and thermal behavior. Raspberry Pi's official 27 W supply supports the relevant 5.1 V/5 A mode [3]; a generic high-wattage label does not guarantee it. The lantern housing carries the compute and battery below the gimbal, with venting and an accessible power switch—there is no face-mounted design. Do not assume long unshielded camera ribbons or joint-crossing cables will work without signal-integrity tests.

14 Build Plan: Color-Coded Tasks and Evidence of Done

[HW]	Hardware (red)
[I/O]	Input/output (blue)
[API]	Pipe/network/integrations (purple)
[VIS]	Visuals/animation (green)
[APP]	Applications (yellow)

Tags and IDs remain legible in grayscale; yellow uses a pale fill with dark gold text. Each task has one primary category; dependencies cross categories. This is a proposed backlog, **not a list of completed features**. "Done" requires saved evidence, not just a plausible screenshot.

14.1 Gate 0: Make a credible, safe bench rig

[HW] H1 / Survey and label the reference geometry
Measure FF corners and sticker offsets, record uncertainty and unique IDs; separate desktop FF, painted solid and proposed stage structure. **Done:** versioned survey and an independent dimension check. **Feeds:** I1, I2.

[HW] H2 / Build the rigid optical carrier on its common gimbal
Mount the camera and projector together on one gimballed carrier; balance the payload, keep joint clearances and lens apertures clear, route cables with service loops, and provide a transport lock. **Done:** the measured camera–projector offset stays invariant over the full tested joint sweep and before/after independent handling. **Feeds:** I3.

[HW] H3 / Verify electrical and thermal budgets
Test exact PD profiles, each rail, startup load, motor idle/slew/dynamic loads, cabling, fuses, mass, enclosure temperature and sustained runtime. **Done:** recorded load/temperature/runtime tests; no brownouts at simultaneous maximum tested output.

[HW] H4 / Establish optical and mechanical operating safety
Exclude targeted blinding/IR countermeasures. Assess the complete projector system under applicable optical standards and venue conditions; provide accessible stop, conservative aiming, gimbal pinch-point guarding and thermal/cable protections—a handheld beam can sweep bystanders. IEC 62471-5 is relevant for image projectors, not a certification granted by this document [4]. **Done:** documented qualified assessment and operating envelope before any handheld or gimballed audience trial.

[I/O] I1 / Inventory and calibrate each optical stream
Save exact K/D model, resolution, preprocessing, lens settings, depth units and hardware identity. Acquire diverse intrinsic-calibration views. **Done:** held-out residual maps and native/resized image consistency; no shared master intrinsics. **Depends:** H1.

[API] N1 / Define a versioned spatial contract
Specify transform direction, units, covariance convention, clocks, world epoch, geometry revision, provenance, expiry and device IDs. Version the carrier camera–projector offsets and, when Route B is used, the joint model: axes, pivots, signs, encoder zeros and limits. These are calibration snapshot metadata, not high-rate control telemetry. **Done:** malformed, stale, wrong-unit and incompatible-revision messages are rejected; synthetic transform round trips agree. **Depends:** agreed conventions in Section 3.

14.2 Gate 1: Prove static multi-device agreement

[I/O] I2 / Establish a room map with independent references
Register fixed cameras to surveyed static markers; register the FF without making it the only live room reference. **Done:** moving the FF leaves an independent room-locked test point unchanged. **Depends:** H1, I1, N1.

[I/O] I3 / Calibrate each projector and rigid offset
Capture coded projected pixels on known surfaces, fit projector models, and estimate camera/projector/carrier transforms on both devices. **Done:** held-out alignment at multiple depths, with the fixed internal camera–projector offset consistent across a joint sweep. **Depends:** H2, I1, I2.

[API] N2 / Fix all-device calibration publication
Stage webcam, projector and Kinect results before one atomic activation; export viewer geometry from that same revision. Fail/report required device skips, include the full outcome in the archive, and refresh an invalid frozen room link rather than promising that FF-only fitting repairs it. **Done:** an injected Kinect failure cannot publish a falsely complete mixed revision; successful exports share the same revision.

[VIS] V1 / Build the alignment and quality view
Draw predicted/detected corners, residual vectors, frame axes, projector coverage, pose age, source IDs and tracking status. Distinguish raw, filtered and predicted data. **Done:** an intentionally wrong K, stale world link and wrong baseline are visibly distinguishable. **Depends:** I2, I3, N1.

[APP] A1 / Ship a guided calibration application
Provide inventory, stationary-target instructions, captured evidence, per-device pass/fail, snapshot review and last-good rollback. **Done:** an unfamiliar operator completes the static setup without editing matrices. **Depends:** N2, V1.

14.3 Gate 2: Move one light without moving the artwork

[I/O] I4 / Measure exposure-to-light timing
Preserve capture timestamps, characterize display buffering and clock offset/skew, and align exposure, joint-encoder and display clocks across static-camera and lantern captures. **Done:** saved timing experiment and latency/jitter distributions, with clock uncertainty. **Depends:** N1.

[I/O] I5 / Implement bounded live carrier tracking
Use direct optical-carrier relocalization (Route A) first; add calibrated carrier-IMU propagation and delayed-state correction only when sensor support exists. The housing-plus-encoder Route B additionally requires a calibrated joint model and world-aligned housing tracking. **Done:** tests for pose direction, gravity, biases, fast motion, single-marker ambiguity, dropout and recovery on each enabled route; Route B stays disabled until I6 and the independent housing–world link are validated. **Depends:** I2, I3, I4.

[I/O] I6 / Measure gimbal state and modes
Record measured joint angles and rates, zero/homing state, joint limits, motor fault flags and per-sample timestamps. No encoder is required for the direct visual-only Route A prototype; this task is required when using the housing route or control supervision. **Done:** slow and fast sweeps and handle-only movement distinguish payload motion from housing motion; missing or stale joint samples invalidate the housing-based route. **Depends:** H2, I4.

[API] N3 / Separate capture, tracking and rendering deadlines
Keep bounded latest-frame queues, isolate CV work, retain timestamps and make tracking age observable independently of render FPS. **Done:** tracker slowdown increases reported age but does not grow an unbounded FIFO or invoke unsupported TD operator work off-thread. **Depends:** I4, N1.

[VIS] V2 / Render the same surface from each current projector pose
Use calibrated per-projector views, distortion mapping, visibility/occlusion and bounded display-time prediction of the carrier pose and surface. Support static and moving projectors. **Done:** moving one rig keeps an independent surface target aligned within the declared dynamic tolerance. **Depends:** I3, I5, N3.

[VIS] V3 / Make tracking failure legible and graceful
Implement tracking/weak/coasting/lost/relative-only states with tested fade/blank behavior, suppression during untrusted slew, homing or controller fault, and no silent coordinate reset. **Done:** occlusion and bumps never label stale light as fully tracked. **Depends:** I5, V1, V2.

[APP] A2 / Demonstrate room-lock versus prop-lock
Let an operator create one room-attached and one FF-attached mark, from a carried housing and from gimbal-only carrier motion. **Done:** moving the FF moves only the latter; moving the housing or the carrier alone keeps room-locked artwork on its surface. **Depends:** I2, V2, V3.

14.4 Gate 3: Share a canvas, not a calibration accident

[API] N4 / Implement session discovery and authenticated control
Separate audience commands from the powerful TD execution bridge; restrict calibration authority and support connection loss/rejoin. **Done:** an unauthorized phone cannot execute Python or change the world epoch. **Depends:** N1, A1.

[API] N5 / Synchronize durable artwork and ephemeral poses
Use unique event IDs, single-writer strokes, append sequence numbers, tombstones, snapshot recovery and latest-valid pose updates. **Done:** duplication, reordering, partitions and reconnect do not duplicate segments or resurrect deleted artwork. **Depends:** N1, N4.

[API] N6 / Integrate a bounded gimbal control interface
Separate commanded targets from measured joint state; publish controller modes, limits, health, sequence numbers and sample age. Accept only allowlisted authenticated commands with rate/position limits and a stop; no arbitrary TD Python execution from a phone. This is a planned API, not an existing CLI. **Done:** stale or reordered telemetry is never treated as fresh pose; a controller fault or homing sequence invokes the documented projection suppression. **Depends:** N1, N4, I6.

[VIS] V4 / Blend overlapping projectors in linear light
Measure gains, color/transfer response and overlap masks; account for occlusion and unavailable emitters. **Done:** an overlap test quantifies brightness/color differences and clipping; a node leaving does not silently overdrive remaining units. **Depends:** I3, V2, N5.

[APP] A3 / Build a collaborative surface-painting scene
Support author identity, brush styles, undo/delete, explicit attachments, offline strokes and clear tracking status. **Done:** two authors edit one persistent canvas and recover from a network interruption without losing existing work. **Depends:** A2, N5.

[APP] A4 / Add phone, Myo and mouse producers honestly
Use tracked phone rays; use Myo/sEMG for consented triggers and calibrated orientation. Provide a declared fixed or tracked ray origin for IMU-only tools; mouse unprojects through a chosen virtual view onto a chosen surface. **Done:** the UI identifies each input's available degrees of freedom; no orientation-only device claims absolute hand position. **Depends:** A3, I5.

14.5 Gate 4: Portable lantern and artistic extensions

[HW] H5 / Package the portable lantern
Provide the vented housing with handle/stand, accessible controls, serviceable joints and cable loops, payload and mass balance, and a reproducible lock/unlock calibration check for transport. **Done:** a carry/park/sweep/pack test preserves the measured carrier calibration. **Depends:** H2–H4, I3–I5, V2.

[VIS] V5 / Design visuals that tolerate measured uncertainty
Use larger features, surface-aware strokes, restrained animation and deliberate transitions; label anamorphic single-view pieces and haze beam sculptures as different modes. **Done:** an artistic review and an error-overlay review agree about which effects remain readable under the tested motion budget. **Depends:** V3, V4.

[APP] A5 / Package rehearsal and performance modes
Provide consent, join/leave, venue setup, recording permissions, accessible alternatives, show cues and an emergency stop. Store no raw sensor data by default beyond the agreed diagnostic policy. Rehearse motor, cable and power faults; no unattended gimbal sweeps around an audience. **Done:** a rehearsal covers power/network/tracking/motor failure and restart. **Depends:** H4, A1–A4, V3.

[APP] A6 / Treat neural prediction as a research experiment
Only after baseline measurement, compare classical prediction with any learned model on held-out motion, including abrupt reversals. **Done:** report error/latency distributions and failure behavior; keep the simpler baseline if learning does not demonstrably improve it. **Depends:** I4, I5, V2; optional, not a prerequisite for art.

15 Presentation Cheat Sheet

Technical term	What to say out loud
Intrinsic calibration	Learn the lens's pixel ruler and bending.
Extrinsic pose	Locate and orient a device relative to a named ruler.
PnP	Fit a known 3D shape to its measured image corners.
Shared world	Agree where the room ruler is, not just where packets go.
Rigid offset	Measure how the camera and light are bolted together.
Homography	A flat-surface shortcut, not a universal 3D warp.
Prediction	A short, uncertain guess about where things will be when light lands.
Spatial pipe	Exchange named, timed, versioned spatial facts and artwork.
Relocalization	Recover the room reference without moving the stored canvas.
Success	People move, collaborate and recover from errors without the light pretending to know more than the sensors do.

The defensible project claim

We are building a shared spatial canvas rendered by independent, calibrated light sources. A measured reference object and additional persistent room constraints connect local sensing to a common world. The system supports both fixed and moving devices, explicitly manages timing and uncertainty, and degrades visibly when evidence is missing.
That is a compelling artistic and computer-science project without claiming zero latency, perfect tracking, or a free-floating hologram.

A Source Register and Traceability

All nine Word documents were supplied from `C:/Users/User/Downloads/ff_docs/drive-download-20261110T090728Z-1-001/`. Their internal version numbers are document labels, not proof of completed implementations.

Key	Exact supplied filename	Contribution / audit focus
S1	`Executive_Summary_Spatial_AR_Headset.docx`	Wearable vision, cost claims, dual projection, optical countermeasures.
S2	`FFv2CalibrationAlgebra&TDArchitecture(pipe_algebra).docx`	Stage scale, localized markers, asynchronous execution, interpolation.
S3	`FFv3PredictiveCalibration&NeuralCanvasArchitecture(pipe_algebra).docx`	Kinematic state, NPU prediction and latency claims.
S4	`FulcrumFrustum(FF)v4_SpatialAlgebra&PerformanceEngine(pipe_algebra).docx`	Artist explanation, transforms, performance-engine synthesis.
S5	`FulcrumFrustum(FF)v5_DynamicTracking&DriftRecovery(pipe_algebra).docx`	Moving FF, bumped cameras, IMU recovery and producer/observer roles.
S6	`FulcrumFrustum(FF)v6_SpatialPipe&Through-SpaceAlignment(pipe_algebra).docx`	Metadata pipe, detect-and-align claims, Myo and heterogeneous input.

Key	Exact supplied filename	Contribution / audit focus
S7	`FulcrumFrustumCalibrationAlgebra(pipe_algebra).docx`	Early projection equations, temporal waves, pose/distortion conflation.
S8	`Ff-writeup.docx`	Artistic rationale, hardware variants, node/control proposals, dual-temple layout, power and shutter claims.
S9	`FormalizedAlgebraofFulcrumFrustum(FF)Calibration.docx`	Detailed geometry, incorrect height and conditioning proof, transform direction inconsistencies, depth and bundle adjustment.

In the audit table, early evolution references are grouped by concept; the original texts sometimes repeat earlier claims under later version labels. In particular the pure "detect-and-align without a solve" claim is explicit in S6, and the pose/distortion conflation is explicit in S7. The wearable head-mounted proposal in S1 and S8 is superseded by the projection-lantern scope; the register entries remain exact historical descriptions of the supplied sources.

Repository evidence inspected

- `tools/fulcrum/fulcrum_spec.json`: measured v1 geometry and corner ordering.
- `tools/fulcrum/ff_calibrate.py`: `phase_webcam`, `phase_kinect`, `export_viewer`, `main`.
- `tools/fulcrum/td_ff_live.py`: stored projector/camera links, `configure_kinect`, `process_frame`, hold/fade and publication.
- `tools/fulcrum/pose_ekf.py`: visual pose/velocity prediction and update.
- `tools/fulcrum/ff_worker.py`: external tracker and shared-memory handoff.
- `tools/fulcrum/ffv2_tracker.py`: separate painted-solid path and camera contract.
- `tools/fulcrum/fulcrum_track.py`: alternative standalone capture owner.
- `tools/td/README.md`: bridge commands and integration constraints.

No hardware calibration or live projection was run for this audit. The numerical examples are deliberately synthetic and do not describe the current physical rig's pose. The script `smoke/xr_doc_extract/check_algebra.py` rechecks those synthetic examples only—it verifies arithmetic and transform conventions, not physical accuracy; the authoritative results are saved at `smoke/xr_doc_extract/algebra_check_results.json`. Likewise, `smoke/xr_doc_extract/check_lantern_kinematics.py` exercises the synthetic lantern kinematics examples only—transform and angle arithmetic, not hardware validation; its authoritative frozen results are at `smoke/xr_doc_extract/lantern_algebra_results.json`.

References

[1] OpenCV, *Perspective-n-Point pose computation*. Object-to-camera transform convention and solver requirements. https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html

[2] OpenCV, *Camera Calibration and 3D Reconstruction*. Projection, distortion and calibration APIs. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

[3] Raspberry Pi, *27 W USB-C Power Supply*, electrical profiles. https://www.raspberrypi.com/products/27w-power-supply/

[4] IEC, *IEC 62471-5:2015: Photobiological safety of lamps and lamp systems—Part 5: Image projectors*, public scope description. https://webstore.iec.ch/en/publication/22639

The OpenCV references describe the API conventions used in Sections 4–7 [1, 2]. They do not certify the custom rig's accuracy.